

Master of Science in Informatics at Grenoble
Master Informatique
Specialization Data Science / Databases and Artificial Intelligence

Cascaded Structured–Semantic Query Processing over Heterogeneous Data

From SUQL and Batched Semantic Joins to Bayesian-Calibrated Two-Level Cascades

Anna

2026

Research and engineering project performed at the intersection of database
systems and large language models (LLMs), MoSIG / Ensimag, Université
Grenoble Alpes

Under the supervision of:
(Supervisor name – to be completed)

Defended before a jury composed of:
(Head of the jury – to be completed)
(Jury member 1 – to be completed)
(Jury member 2 – to be completed)

Abstract

Answering natural-language questions that mix structured predicates (e.g. release year, film duration, genre) with unstructured predicates over free text (e.g. “the reviews are strongly negative”) requires a query engine that can call a large language model (LLM) as a relational operator. Two execution strategies are commonly used: a *predicate-pushdown* strategy that pushes structured filters into SQL and evaluates the semantic predicate with one LLM call per candidate row (the SUQL-style), and a *batched-join* strategy that packs many rows into a single LLM call and asks the model to execute the whole predicate over a chunk of serialized rows at once (the Trummer-style batch join). We show empirically, on ten held-out, genre-diverse questions against a controlled, auditable 100-movie pool per question (drawn from a $\sim 25k$ -row IMDb corpus with paired structured metadata and free-text reviews, each question run ten times per method), that the row-wise strategy is accurate but slow (117.1 calls and 115.6s per question) while the batched strategy is faster (96.9 calls, 71.3s) but visibly less accurate (F1 0.444 vs. 0.678). We then utilize a *two-level cascade* to introduce a fast, yet quality preserving pipeline that adds a structured SUQL-style filtering before the Trummer-style batch join block and finally a cheap, uncertainty-aware pre-filter in front of either strategy: rows (or batches) for which the cheap stage is confident are resolved immediately, and only the ones that fall inside a Bayesian-calibrated ambiguity interval are escalated to the expensive stage. Cascading the row-wise strategy (SUQL V1) improves F1 by 2.7% over the row-wise baseline using about 40% fewer calls. Cascading the batched strategy (Trummer V1) is the best overall design point: it reaches F1 0.757 (+8.8% over SUQL V1, +70.4% over the batched baseline) and the best precision of all four variants (+14.2% over the row-wise cascade), while using only 1/26 of the wall-clock time and 1/7 of the LLM calls of the row-wise cascade, at the cost of a small ($\approx 7\%$) recall regression relative to it. We give the full architecture, pseudocode, prompts, and a Beta-posterior formalization of the calibrated cascade threshold that explains this cost–quality curve.

Acknowledgement

I would like to express my sincere gratitude to my supervisor and to the members of the jury for their invaluable comments and guidance throughout this project.

Résumé

Les questions en langage naturel posées sur des ensembles de données hétérogènes mélangeant colonnes structurées (année de sortie, genre, note) et champs textuels libres (critiques) ne peuvent être résolues ni par le seul SQL, ni par un grand modèle de langage (LLM) seul à un coût raisonnable. Ce rapport conçoit, implémente et évalue une famille de moteurs d’exécution de requêtes structurées–non structurées (dans l’esprit de SUQL) sur un jeu de données IMDb d’environ 25 000 lignes, et étudie le compromis entre deux stratégies d’exécution : le *predicate pushdown* ligne par ligne (SUQL) et la *jointure sémantique groupée* (style Trummer). Nous introduisons une *cascade à deux niveaux*, calibrée par une borne

crédible bayésienne (loi Bêta) sur les log-odds à la manière de Stretto, qui améliore simultanément le coût et la qualité par rapport aux deux références.

Contents

Abstract	i
Acknowledgement	i
Résumé	i
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	1
1.3 What Existed Before This Project	2
1.4 Contributions of This Report	3
1.5 Report Roadmap	3
2 Background and Formal Query Model	5
2.1 Related Work	5
2.1.1 SUQL	5
2.1.2 ELEET	5
2.1.3 Stretto	5
2.1.4 Positioning	6
2.2 Formal Query Model	6
2.2.1 Log-odds representation of a semantic decision	7
3 Baseline Architectures and Implementation	9
3.1 System Overview	9
3.2 SUQL Baseline: Row-Wise Predicate Pushdown	10
3.3 Trummer Baseline: Adaptive Two-Collection Block Join	12
3.4 Shared Evaluation Harness	14
4 Two-Level Cascading: Architecture, Calibration, and Theory	15
4.1 Architecture	15
4.2 Per-Variant Instantiation	15
4.2.1 SUQL V1: cascaded predicate pushdown	15
4.2.2 Trummer V1: cascaded, structured, batched join	17
4.3 Calibrating the Cascade	19

4.3.1	Why not a hand-picked threshold?	19
4.3.2	Setting up the posterior	19
4.3.3	The credible bound, and a subtle quantile pitfall	19
4.3.4	Choosing the band	20
4.4	Operator Ordering	20
4.5	Theoretical Cost Analysis of Calibrated Cascading	20
4.5.1	Setup	21
4.5.2	Expected-cost crossover	21
4.5.3	High-probability realized-cost dominance	22
4.5.4	Worked instantiation from the measured system	22
5	Experimental Setup and Results	25
5.1	The 10-Question Benchmark Suite	25
5.1.1	Dataset construction	25
5.1.2	The ten questions	26
5.1.3	Run configuration	26
5.2	Aggregate Results	26
5.3	Per-Question Breakdown	27
5.4	Benchmark Visualizations	28
5.5	Cross-Domain Validation: Amazon Data	29
5.5.1	IMDb vs. Amazon Fashion: Matched Suite, Same Model	30
5.5.2	Scaling and Reliability Validation	32
5.5.3	Qwen vs. Gemma on Amazon	34
	Remaining scope.	36
5.6	Discussion	37
6	Conclusion and Future Work	45
6.1	Summary	45
6.2	Engineering Lessons	45
6.3	Current Implementation Status	46
6.4	Future Work	46
6.4.1	Batch SUQL V1’s cheap stage	46
6.4.2	Port Beta-posterior calibration to Trummer V1	46
6.4.3	Amortize the calibration budget across repeated queries	47
6.4.4	The KV-cache-aware execution ladder	47
6.4.5	Extending the selectivity/difficulty grid	47
6.4.6	Early termination for LIMIT queries	47
A	Prompts and Reproduction Reference	49
A.1	Complete Prompt Reference	49
A.2	Calibration Reference Values	52
A.3	Reproduction: Running the Benchmarks Locally	52
	Bibliography	55

Introduction

1.1 Motivation

Modern data pools are rarely purely relational. A movie catalogue such as IMDb, for instance, pairs a structured relation – title, release year, genre, average rating, cast – with a large body of free-text evidence: plot summaries and, above all, user reviews. A realistic information need over such a pool routinely mixes both modalities in a single question:

“Which movies released in 1998 have reviews expressing an overall negative, critical, or strongly unfavorable opinion of the movie?”

The first half of this question – `year = 1998` – is a textbook relational predicate: it is answered exactly, cheaply, and instantly by a `WHERE` clause against an indexed column. The second half – *“expressing an overall negative, critical, or strongly unfavorable opinion”* – is a semantic predicate over unstructured text that no amount of indexing or exact-match SQL can express. It requires either a learned text classifier or, at the level of accuracy and flexibility we target here, a large language model (LLM) invoked as a query operator.

The engineering problem this report addresses is therefore not “can an LLM answer this question” – it clearly can, given enough context and enough calls – but rather: *how do we build a query engine that answers such questions correctly, while keeping the number of LLM calls and the wall-clock latency as low as possible?* This is a cost-vs-quality optimization problem with the same flavor as classical query optimization (operator ordering, selectivity estimation, pushdown), except that one of the “operators” is a probabilistic, expensive, non-relational function: the LLM.

1.2 Problem Statement

Formally, we are given:

- a structured relation R with schema $(id, year, genre, rating, cast, \dots)$, $|R| \approx 25,000$ rows;
- a free-text relation $D(id, review)$, in a one-to-one (or one-to-few) correspondence with R on id ;

- a natural-language query Q that decomposes (possibly only implicitly) into a structured predicate σ_S over R 's columns and a semantic predicate σ_U over D 's text, joined conjunctively: $Q \equiv \sigma_S \wedge \sigma_U$.

The desired output is the JSON-serializable result set $\{r \in R \bowtie D : \sigma_S(r) \wedge \sigma_U(r)\}$, and the desired *system* property is that it produce this set (i) as close as possible to a hand-verified gold answer set (measured by precision, recall, F1), (ii) with as few LLM calls as possible, and (iii) with as low wall-clock latency as possible. These three objectives are in tension, and the remainder of this report is an account of how far a principled combination of classical query-optimization ideas (predicate pushdown, operator reordering, selectivity-aware short-circuiting) and modern LLM-serving ideas (batched invocation, confidence calibration, cascaded inference) can push the achievable trade-off surface.

All code, prompts, and benchmark data referenced in this report are drawn from the project repository at <https://github.com/AnnaRemi/ENSIMAG-AI-M2-final-project>, branch `agent/consolidate-four-method-benchmarks`. Experiments (Chapter 5) are run against controlled, 100-row-per-question subdatasets drawn from the $\sim 25,000$ -row canonical corpus above, rather than against the full corpus directly, so that every question has a known, auditable gold set of a fixed size (Section 5.1).

1.3 What Existed Before This Project

Two execution strategies were already implemented and benchmarked before the work reported here, and both are treated as *baselines* rather than as strawmen: they are each the right choice for some regime of the problem, and understanding exactly why is what motivates the cascaded design of Chapter 4.

SUQL baseline. Following the design of SUQL [1], the structured predicate σ_S is compiled to SQL and evaluated first, and the semantic predicate σ_U is evaluated by issuing one `answer()` call per surviving row to an LLM. Quality is high because every row is judged individually and in full context, but the number of LLM calls equals $|R_{\sigma_S}|$ – the size of the structurally filtered relation – so the strategy does not scale gracefully when the structured predicate is not very selective.

Trummer baseline. The structured relation and the review relation are kept as two *separate* collections – deliberately, by design, with *no* SUQL-style structured pushdown – and the LLM itself is asked to join a block of one collection against a block of the other while simultaneously applying the semantic predicate, an adaptive block-pair join over the *entire* candidate pool. This is far cheaper in calls and wall time than row-wise judgment, but is measurably less accurate: the model must recover row identity from block text and perform the join itself, and errors (id drift, incomplete coverage of a block's decisions) compound without any structural narrowing of the pool it has to cover.

Both baselines are described in full, with architecture diagrams and pseudocode, in Chapter 3.

1.4 Contributions of This Report

1. **An apples-to-apples empirical comparison** of row-wise predicate pushdown and adaptive block-join execution, on ten held-out, genre-diverse questions drawn from the canonical $\sim 25\text{k}$ -row IMDb corpus, each run ten times per method, with matched result sets and a shared metric suite (precision, recall, F1, wall time, LLM calls per question) – Chapter 5.
2. **A two-level cascaded execution architecture**, applicable on top of either baseline, in which a cheap first-stage scorer resolves the majority of units of work (rows or batches) immediately, and only escalates the units whose Stage 1 output falls inside a calibrated ambiguity band to the expensive second-stage operator – Chapter 4.
3. **A closed-form Bayesian calibration rule** for the escalation band, derived from a Beta posterior over the cheap stage’s empirical recall/precision on a held-out calibration set, following the backend-agnostic half of Stretto’s contribution [2] – Section 4.3. This rule is fully implemented for SUQL V1; Trummer V1 currently uses a simpler point-estimate threshold search, and unifying the two is identified as near-term future work (Section 4.2.2).
4. **An operator-reordering rule**, inherited from ELEET’s cascade optimizer [3] and from classical selectivity-ordered filter placement, used to decide in which order the structural filter, the batched semantic pass, and the verification step should run – Section 4.4.
5. **A formal proof that calibrated cascading beats an all-expensive baseline for large enough candidate pools** – both in expectation (a finite, closed-form crossover point n^*) and, via a Hoeffding concentration bound, with probability approaching 1 exponentially fast in the pool size – together with a worked instantiation from the measured system that *predicts* the empirical asymmetry between SUQL V1 and Trummer V1 rather than merely describing it after the fact – Section 4.5.
6. **A full empirical characterization** of the four resulting design points (SUQL baseline/V1, Trummer baseline/V1), showing that the cascaded batched-join design (Trummer V1) dominates on cost, F1, *and* worst-case robustness across the ten held-out questions – Chapter 5.
7. **An honest account of engineering pitfalls** encountered along the way – an off-by-quantile error in the Bayesian bound, hardcoded gold-standard counts, a Trummer baseline that turned out (on inspection of the real implementation) to skip structural pushdown entirely rather than merely defer it, and a missing early-termination optimization for LIMIT queries – because these are as much a part of building a reliable LLM-as-operator system as the architecture itself.

1.5 Report Roadmap

Chapter 2 reviews the three pieces of prior work this project builds most directly on – SUQL, ELEET, and Stretto – and fixes the formal query model used throughout. Chapter 3 describes the two baseline architectures in detail, with diagrams, pseudocode, and the exact prompts used,

including a correction of an earlier draft’s mischaracterization of the Trummer baseline’s join structure. Chapter 4 is the technical core of the report: it develops the two-level cascade, its calibration procedures, the operator-ordering rule, and – new in this revision – a full theoretical cost analysis proving when and why calibrated cascading beats an all-expensive baseline, mathematically and in pseudocode, for both the SUQL V1 and Trummer V1 variants. Chapter 5 presents the ten-question experimental setup and results. Chapter 6 concludes and lays out the remaining roadmap, in particular the KV-cache-aware execution ladder (Stretto’s second contribution) that is deferred pending a backend migration, and the near-term engineering changes the theory of Chapter 4 points to directly. Appendix A collects the full system prompts, verbatim from the repository, and the actual commands used to reproduce every experiment in this report.

Background and Formal Query Model

2.1 Related Work

2.1.1 SUQL

SUQL [1] extends SQL with a free-text predicate function, written `answer() (text, question)`, that is backed by an LLM and returns a boolean (optionally with a short justification). Its central systems argument, which this project adopts wholesale, is that *predicate pushdown* – executing every structured predicate *before* any semantic predicate – is the dominant lever for controlling LLM invocation cost, because it shrinks the candidate set the expensive operator ever has to see. SUQL treats `answer()` as an opaque, per-row, non-batchable user-defined function (UDF); it does not, in its base form, address the cost of the semantic operator itself once the candidate set is fixed. That is exactly the gap this project’s cascading contribution targets.

2.1.2 ELEET

ELEET [3] generalizes the single-source assumption of SUQL: a logical scan is modeled as a *multi-modal operator* that can pull evidence for the same entity from more than one physical source – structured columns in one table, free text in another – and fuse it before a selection is applied. This is directly relevant to our setting, where the IMDb structured metadata and the review text are joined from two logically separate sources rather than pre-materialized into one table. ELEET also formalizes an *operator cascade*: a sequence of selection operators, each with an associated cost and selectivity, that can be reordered to minimize expected total cost, and each parametrized by a *log-odds* score rather than a hard boolean, which is the representation this project’s cascade calibration is built on (Section 2.2.1).

2.1.3 Stretto

Stretto [2] is the most recent piece of prior work this project draws on, and contributes two largely independent ideas:

1. **Bayesian threshold calibration.** Rather than hand-tuning the log-odds cutoff at which a semantic operator’s decision is trusted without further verification, Stretto derives a *credible interval* on the operator’s precision and recall at any candidate threshold from

a Beta posterior fit to a calibration set, and picks the threshold(s) that certify a target quality floor at a target confidence level. This procedure is a pure post-processing step over whatever log-odds score the cheap stage already produces; it requires no change to how that score is computed, and is therefore **backend-agnostic**.

2. **Cache-aware execution ladder (Expected Attention Press)**. Stretto’s second contribution reorders and prunes cascade stages at a much lower level: it reuses attention key/-value (KV) cache state *across* cascade stages, so that escalating an ambiguous item from a cheap to an expensive stage does not require recomputing attention over the shared context from scratch. This requires direct access to the serving engine’s attention state (e.g. via vllm’s internal APIs), and is **not portable** to an Ollama-served backend without replacing the serving layer.

This project builds on contribution (1) only; contribution (2) is deferred to future work (Section 6.4.4) pending a migration away from Ollama. Neither Stretto nor the systems it builds on state a formal condition for *when* a calibrated cascade is guaranteed to reduce cost relative to an all-expensive baseline; Section 4.5 closes this gap for the two-level cascade studied here, with a finite crossover point and a concentration guarantee, instantiated against measured system constants.

2.1.4 Positioning

Table 2.1 summarizes how this project’s contributions relate to the three pieces of prior work above.

Source	What we adopt / extend
SUQL	Predicate-pushdown discipline; <code>answer()</code> as the expensive, per-row semantic operator inside the cascade’s second stage.
ELEET	Multi-modal operator abstraction for the two-source join; log-odds parametrization of a semantic filter; operator-cascade cost/selectivity reordering rule (Section 4.4).
Stretto	Bayesian Beta-posterior calibration of the cascade’s escalation band (Section 4.3); we do not (yet) adopt the KV-cache execution ladder.

Table 2.1: How this project’s cascaded design relates to prior work.

2.2 Formal Query Model

We fix the following notation, used throughout the rest of the report.

- R : the structured relation, schema $(id, year, genre, rating, cast, \dots)$.
- D : the unstructured relation, schema $(id, review)$.
- σ_S : a structured predicate over R ’s columns, compilable to a SQL WHERE clause.

- σ_U : a semantic predicate over D 's free text, expressed in natural language and evaluated via $\text{answer()}(\text{review}, \sigma_U)$.
- $R_{\sigma_S} = \{r \in R : \sigma_S(r)\}$: the structurally filtered relation.
- $s = |R_{\sigma_S}|/|R|$: the *selectivity* of the structured predicate.
- $\text{gold} \subseteq R \bowtie D$: a hand-verified gold answer set for a given query Q , used to compute precision, recall, and F1.

The target output of any execution strategy is

$$\widehat{\text{ans}} = \{r \in R_{\sigma_S} \bowtie D : \sigma_U(r) \text{ holds}\},$$

and every strategy studied in this report is judged on how closely $\widehat{\text{ans}}$ tracks gold (precision/recall/F1), and on how many LLM calls and how much wall-clock time it takes to produce $\widehat{\text{ans}}$.

2.2.1 Log-odds representation of a semantic decision

Any operator (cheap or expensive) that evaluates σ_U against a unit of work u (a row or a batch of rows) is modeled as returning a probability $p = f(u, \sigma_U) \in (0, 1)$ that u satisfies σ_U , rather than a bare boolean. We work in *log-odds* space,

$$\ell = \text{logit}(p) = \log \frac{p}{1-p} \in \mathbb{R},$$

for two reasons that matter operationally, not just aesthetically:

- **Additivity.** Under (approximately) independent evidence combination, log-odds from multiple signals add, which makes it natural to combine a cheap-stage score with, e.g., a self-consistency vote or an embedding-similarity prior, without renormalizing probabilities by hand at every step.
- **Symmetric decision point.** $\ell = 0$ corresponds to $p = 0.5$, the natural undecided point, and $|\ell|$ is a monotone measure of confidence in either direction, which is exactly the quantity the cascade thresholds on (Chapter 4).

Baseline Architectures and Implementation

This chapter describes the two baseline execution strategies in full: their architecture, their pseudocode, the exact prompts issued to the LLM, and the implementation layer (SQL parsing, the multi-modal operator abstraction, and the evaluation harness) shared by both. Unlike earlier drafts of this report, every prompt and every architectural claim in this chapter is taken verbatim or paraphrased directly from the project repository, branch `agent/consolidate-four-method-benchmarks`: <https://github.com/AnnaRemi/EN-SIMAG-AI-M2-final-project/tree/agent/consolidate-four-method-benchmarks>. Where an earlier draft of this report guessed at an implementation detail that turned out to differ from the real code (most notably the Trummer baseline’s join structure, corrected in Section 3.3), the correction is noted explicitly rather than silently.

3.1 System Overview

Figure 3.1 shows the overall system: a natural language query is decomposed into a structured predicate σ_S and a semantic predicate σ_U (predicate extraction), the structured part is compiled to SQL and run first, and the semantic part is then evaluated by one of the four strategies described in this chapter and the next.

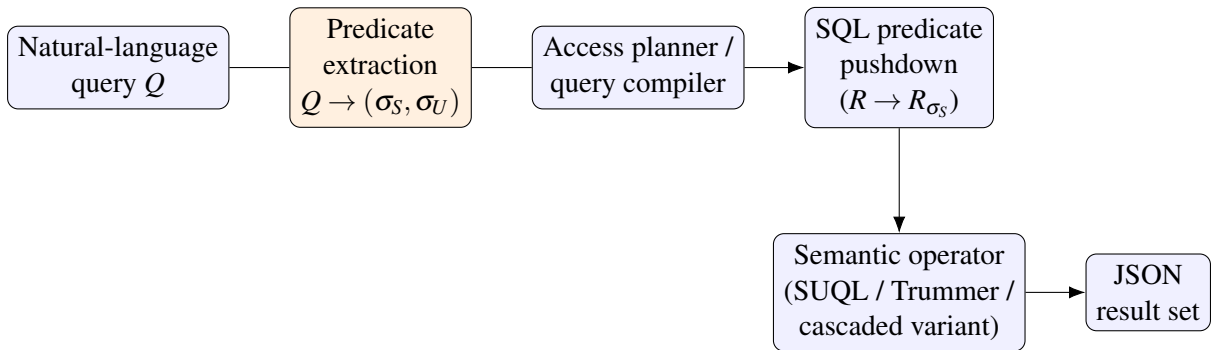


Figure 3.1: End-to-end system: NL query $\rightarrow$ predicate extraction $\rightarrow$ SQL pushdown $\rightarrow$ plug-able semantic-operator strategy $\rightarrow$ JSON output. The four strategies compared in this report (SUQL baseline/V1, Trummer baseline/V1) differ in the “semantic operator” box, and – importantly, see Section 3.3 – in whether the SQL-pushdown box is applied at all before that operator runs.

For SUQL (baseline and V1), predicate extraction is itself an LLM call (`nl_to_suql`, a few-shot semantic parser that compiles the NL question directly into a SUQL query – plain SQL `WHERE` clauses for structured columns, plus `answer()`/`summary()` free-text predicates), and the parser’s own system prompt explicitly instructs the model to place structured predicates before `answer()` in the `WHERE` clause “for efficiency.” The structural SQL is then stripped out and executed on an in-memory SQLite instance (“`litesql`”) *before* any `answer()` call is issued – Section 3.2 gives the exact pipeline. For Trummer (baseline and V1), predicate extraction and structural filtering are handled differently, and, critically, *not at all* in the baseline – see Section 3.3.

3.2 SUQL Baseline: Row-Wise Predicate Pushdown

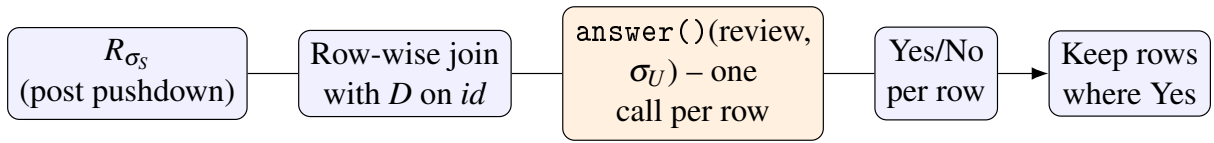


Figure 3.2: SUQL baseline: after SQL pushdown, every surviving row is joined to its review and judged by one `answer()` call.

Pipeline. The engine (`project SUQL/baseline/suql_engine.py`) implements the SUQL paper’s own pipeline in five steps: (1) split the compiled SUQL query into a structural SQL part and a set of `answer()`/`summary()` free-text predicates; (2) run the structural SQL on an in-memory SQLite table; (3) for every row that survives, evaluate each `answer()` predicate with one LLM call, short-circuiting the remaining predicates for a row the moment one of them returns No; (4) keep only rows that pass every `answer()` predicate; (5) compute `summary()` values for any requested summary columns. Row-level results are cached in-process on (`review_hash`, `question`) to avoid re-querying identical pairs.

Cost model. For a single `answer()` predicate, the number of semantic-filter LLM calls is exactly $|R_{\sigma_S}|$, one per structurally-surviving row; wall-clock time is this many calls executed serially against a single Ollama-served model instance. In practice the harness’s `llm_calls` metric (Section 3.4) counts *every* LLM prompt the pipeline issues for a question – including the one-time semantic-parsing call and any `summary()` calls – so the measured call count is a modest constant multiple of $|R_{\sigma_S}|$ rather than exactly $|R_{\sigma_S}|$; Chapter 5 reports both the raw measured totals and, in Section 4.5, a “lean” one-call-per-row idealization used for the theoretical cost analysis.

Quality. Because every row is judged individually, with the full review text in context and no competing rows to distract the model, this strategy is close to the achievable quality ceiling for a given base model; empirically (Chapter 5) it is a solid, consistent mid-table performer among the four studied variants.

Listing 3.1: `answer()` system prompt, verbatim from `project SUQL/baseline/suql_engine.py` (SUQL baseline).

```

You are a recall-first semantic filter for movie reviews.

Decide whether the review provides evidence that the movie

```

Algorithm 1 SUQL baseline: row-wise predicate pushdown

Require: structured relation R , review relation D , structured predicate σ_S , semantic predicate

σ_U

```
1:  $R_{\sigma_S} \leftarrow \text{EXECUTESQL}(\text{SELECT } * \text{ FROM } R \text{ WHERE } \sigma_S)$ 
2:  $result \leftarrow \emptyset$ 
3: for  $r \in R_{\sigma_S}$  do
4:    $review \leftarrow D[r.id]$  ▷ join on id
5:   if  $(review, \sigma_U) \in cache$  then
6:      $b \leftarrow cache[(review, \sigma_U)]$ 
7:   else
8:      $b \leftarrow \text{LLMCALL}(\text{answer}(review, \sigma_U))$  ▷ 1 LLM call
9:      $cache[(review, \sigma_U)] \leftarrow b$ 
10:  end if
11:  if  $b = \text{YES}$  then
12:     $result \leftarrow result \cup \{r\}$ 
13:  end if
14: end for
15: return  $result$ 
```

satisfies the question.

Recall is the primary objective, but do not accept every review:

- Answer YES when there is any concrete review-specific evidence: direct wording, a synonym/paraphrase, a described example, or a reasonable implication.
- Give borderline evidence the benefit of the doubt when it specifically bears on the condition. The evidence need not use the question's exact words.
- This is evidence retrieval, not sentiment scoring: an explicit mention still counts when it is negated, qualified, quoted, or used critically.
- Answer NO when there is no review-specific support, the connection is only the movie's genre/topic, the text is generic praise or criticism, or the review never discusses the condition itself.
- Do not infer evidence merely because the condition is not contradicted.

Return exactly YES or NO. Do not include reasoning, punctuation, or extra text.

Two design choices in this prompt matter for the cost/quality discussion in later chapters. First, it is explicitly *recall-first*: it instructs the model to give ambiguous evidence the benefit of the doubt, which biases the baseline toward higher recall at some precision cost. Second, it asks for a bare YES/NO token rather than a JSON object or a justification string, which keeps the completion short (a handful of output tokens) and avoids the JSON-parsing failure mode that a richer response schema would introduce.

3.3 Trummer Baseline: Adaptive Two-Collection Block Join

Correction relative to earlier drafts. An earlier draft of this report described the Trummer baseline as operating on the *structurally pre-filtered* relation R_{σ_S} , serialized into a single header-plus-rows chunk. This is incorrect for the actual implementation (project Trummer/baseline/trummer_join/), and the correction matters: the Trummer baseline is, by design, the *no-pushdown* reference point in this study. Its own README states plainly that it uses “no SUQL-style structured pruning or cascade”; the structured relation and the review relation are kept as two *separate* logical collections, and the LLM itself is asked to perform the join, mirroring the two-source, multi-modal retrieval setting that ELEET’s operator abstraction targets (Section 2.2).

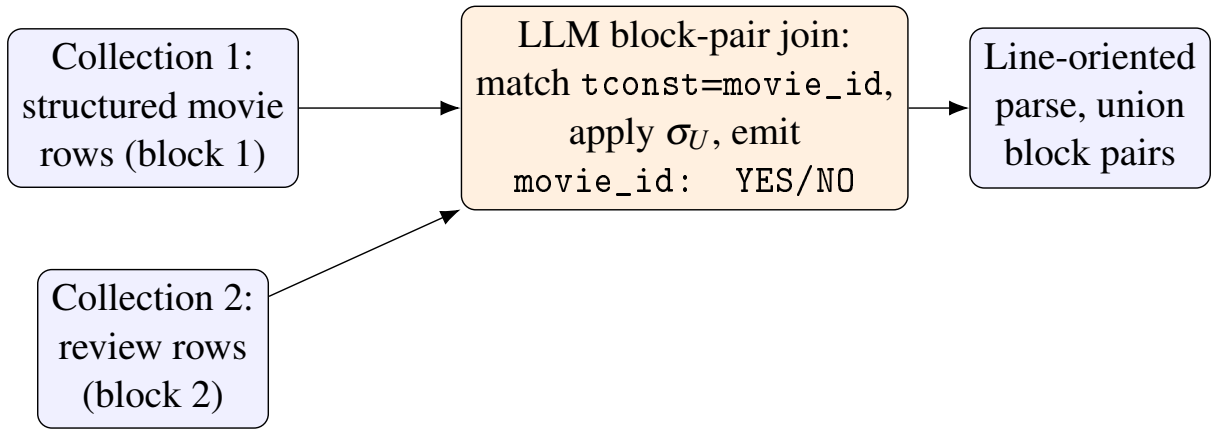


Figure 3.3: Trummer baseline: no structural pushdown. The full candidate pool is split into blocks of each collection independently; the LLM is asked to join a block of Collection 1 against a block of Collection 2 *and* apply the semantic predicate in the same call.

Pipeline. The candidate movies (Collection 1) and their reviews (Collection 2) are each partitioned into blocks via `partition_records`, with block size chosen by an `optimal_block_size` routine (a port of an existing `llmjoin` block-size tuning utility). `join_two_blocks` sends one block of each collection to the LLM in a single prompt (Listing 3.2) and parses a line-oriented `movie_id: YES / movie_id: NO` response with a tolerant regex-based parser (`parse_pairs/parse_decisions`), so execution does not depend on provider-specific structured-output support. `adaptive_join` (rather than the simpler `block_join`) additionally checks response coverage (`has_complete_decision_coverage`) and adjusts block pairing when a response fails to decide every pair it was asked about – the “adaptive” half of the name.

Cost model. Because there is no structural pushdown, the number of LLM calls scales with the number of block *pairs* needed to cover the *entire* candidate pool (up to 100 rows in the 10-query benchmark of Chapter 5, not just the 40 that satisfy σ_S), and grows further whenever a block pair’s response fails coverage and must be retried or re-partitioned – this is the main reason the Trummer baseline’s call count is both higher, on average, than a naive $|R|/(\text{block size})$ estimate would suggest, and highly variable across questions (Chapter 5 reports a range from 30 to 268 calls across the ten held-out questions).

Quality. Two systematic error sources appear, both traceable to the fact that the model must recover row identity and column structure from block text and perform the id-matching join itself, without any guarantee of covering every pair in one pass:

1. **Join/parsing errors.** A block pair may return `movie_ids` that do not exactly match any row in Collection 1 (drift in how the model echoes the id), which the tolerant parser discards rather than guesses.
2. **Incomplete coverage.** The model may decide only a subset of the pairs in a block before truncating or drifting, which – absent structural pushdown to shrink the pool the join has to cover in the first place – is the dominant driver of the Trummer baseline’s low recall (Chapter 5).

Algorithm 2 Trummer baseline: adaptive two-collection block join

Require: structured relation R (*not* pre-filtered by σ_S inside this operator), review relation D , semantic predicate σ_U

- 1: $B_1 \leftarrow \text{PARTITIONRECORDS}(R, \text{OPTIMALBLOCKSIZE}(R))$
- 2: $B_2 \leftarrow \text{PARTITIONRECORDS}(D, \text{OPTIMALBLOCKSIZE}(D))$
- 3: $\text{result} \leftarrow \emptyset$
- 4: **for** $(b_1, b_2) \in B_1 \times B_2$ **as scheduled by** ADAPTIVEJOIN **do**
- 5: $\text{resp} \leftarrow \text{LLMCALL}(\text{join-prompt}(b_1, b_2, \sigma_U))$ ▷ 1 LLM call per block pair
- 6: **if** $\neg \text{HASCOMPLETECOVERAGE}(\text{resp}, b_1)$ **then**
- 7: ADJUSTBLOCKPAIRING() ▷ retry/re-partition
- 8: **end if**
- 9: $\text{pairs} \leftarrow \text{PARSEPAIRS}(\text{resp}, b_1, b_2)$
- 10: $\text{result} \leftarrow \text{result} \cup \{r_1 : (r_1, r_2) \in \text{pairs}\}$
- 11: **end for**
- 12: **return** $\text{result} \cap R_{\sigma_S}$ ▷ σ_S applied post hoc, if at all, by the harness

Listing 3.2: Block-join prompt, verbatim from project Trummer/baseline/trummer_join/operators.py (create_prompt).

```
Find every movie in Collection 1 that has a review in
Collection 2 satisfying this predicate:
{predicate}
Act as a recall-first semantic filter, while requiring
review-specific evidence.
- Answer YES for direct wording, synonyms/paraphrases,
  described examples, or reasonable implications.
- Give borderline evidence the benefit of the doubt when it
  specifically bears on the predicate.
- Count explicit mentions even when negated, qualified,
  quoted, or critical; retrieve evidence rather than score
  sentiment.
- Answer NO when support is absent, generic, based only on
  genre/topic, or never discusses the predicate itself.
- Lack of contradiction alone is not evidence.
Collection 1 contains structured movie rows. Collection 2
contains review chunks.
```

```

A review belongs to a movie only when tconst is exactly
equal to movie_id.
Return one decision per line as: movie_id: YES or
movie_id: NO.
Do not include confidence, evidence, explanations, or prose.
Copy each movie_id exactly from Collection 1.
Do not add prose, JSON, markdown, or a completion marker.

Collection 1:
{block_1_rows}

Collection 2:
{block_2_rows}

Decisions:

```

Note the shared evidence-retrieval language between this prompt and the SUQL baseline’s `answer()` prompt (Listing 3.1): both explicitly frame the task as recall-first evidence retrieval rather than sentiment scoring, and both instruct the model to count negated, qualified, or critical mentions as evidence. This is a deliberate, shared design choice across both projects in the repository, and it means the quality gap between the two baselines (Chapter 5) is attributable to the *execution strategy* – row-wise judgment with pushdown vs. block-wise LLM-driven join without pushdown – and not to a difference in how “satisfies the predicate” is defined for the underlying model.

3.4 Shared Evaluation Harness

All four variants (both baselines here, and both cascaded V1 variants of Chapter 4) are evaluated with the same harness (`benchmarks/shared/scripts/`):

- **Gold set derivation.** Rather than hardcoding an expected positive count (an early version of the evaluation script fixed `actual_p = 11` for a specific query and had to be corrected), the current benchmark suites derive ground truth by construction: every question’s candidate pool is assembled from annotated rows whose `structured_match` and `semantic_label` columns are known in advance (Section 5.1), and `ground_truth` is their logical conjunction. This keeps the confusion-matrix computation (TP, FP, FN, TN) parameterized by the actual question and its annotated pool, rather than by a magic constant, and additionally makes every label auditable via an `evidence_excerpt` and an `annotation_rationale`.
- **Metrics.** Precision, recall, F1 against the gold set; wall-clock time per question; total LLM calls per question, plus (for the cascaded variants) the cheap/expensive split of both calls and wall time – computed identically for all four variants and averaged over repeated runs (Section 5.1).
- **Implementation isolation.** Each method’s engine module is imported fresh per run with its own environment configuration (model names, API base, cascade parameters), and its in-process cache is cleared before each run, so repeated executions are not silently accelerated by stale cached answers from a previous method or question.

Two-Level Cascading: Architecture, Calibration, and Theory

This chapter is the technical core of the report. It develops, precisely and end to end, the two-level cascade that is layered on top of each baseline to produce SUQL V1 and Trummer V1: the architecture (Section 4.1), the per-variant instantiation with the real prompts and parameters used by the implementation (Section 4.2), the calibration procedure that chooses the escalation threshold(s) from data (Section 4.3), the operator-reordering rule that decides in which sequence the cheap and expensive stages should run (Section 4.4), and – new in this revision – a formal proof that a calibrated cascade’s expected and realized cost (both LLM calls and wall-clock time) is lower than an all-expensive baseline’s once the candidate pool is large enough (Section 4.5).

4.1 Architecture

The cascade treats the semantic predicate σ_U not as a single opaque call, but as a two-stage pipeline over each *unit of work* u (a row, for SUQL V1; a row scored inside a batched pass, for Trummer V1):

1. **Stage 1 (cheap).** A cheap scorer $f_1(u, \sigma_U) \rightarrow p \in (0, 1)$ (or, for Trummer V1, a direct three-way label) estimates whether u satisfies σ_U , using a smaller/faster model than Stage 2.
2. **Stage 2 (expensive).** A single-row `answer()`-style call, invoked *only* on the subset of units whose Stage 1 output falls inside a calibrated ambiguity band.

4.2 Per-Variant Instantiation

4.2.1 SUQL V1: cascaded predicate pushdown

After structural pushdown, `project SUQL/v1/scorer.py` scores every surviving row with a *single-token, logprob-based* cheap probe: the model is asked to emit exactly one token (1 for

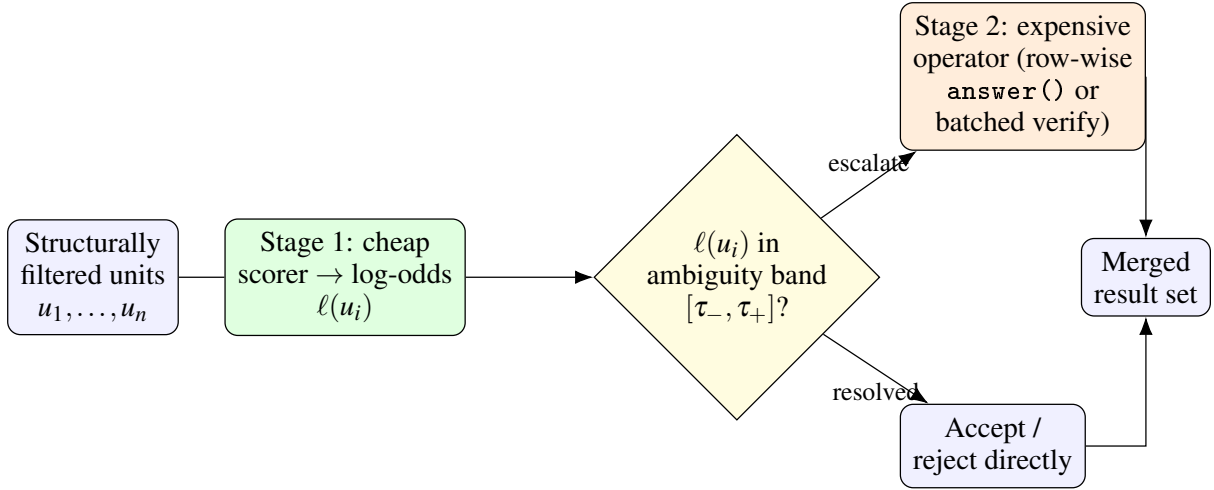


Figure 4.1: Two-level cascade shared by SUQL V1 (unit = row, $B = 1$) and Trummer V1 (unit = row, scored inside a batched pass of $B = 8$ rows, escalated individually).

Algorithm 3 Two-level cascaded semantic filter

Require: rows R_{σ_S} (post structured pushdown), predicate σ_U , cheap scorer f_1 , expensive operator f_2 , calibration set C , target quality Q^*

- 1: $(\tau_-, \tau_+) \leftarrow \text{CALIBRATEBAND}(f_1, C, Q^*)$ ▷ Section 4.3
 - 2: $\text{Accept}, \text{Reject}, \text{Escalate} \leftarrow \emptyset, \emptyset, \emptyset$
 - 3: **for** $u \in R_{\sigma_S}$ **do**
 - 4: $\ell \leftarrow \text{logit}(f_1(u, \sigma_U))$
 - 5: **if** $\ell \geq \tau_+$ **then**
 - 6: $\text{Accept} \leftarrow \text{Accept} \cup \{u\}$
 - 7: **else if** $\ell \leq \tau_-$ **then**
 - 8: $\text{Reject} \leftarrow \text{Reject} \cup \{u\}$
 - 9: **else**
 - 10: $\text{Escalate} \leftarrow \text{Escalate} \cup \{u\}$
 - 11: **end if**
 - 12: **end for**
 - 13: **for** $u \in \text{Escalate}$ **do** ▷ Only ambiguous units reach Stage 2
 - 14: **if** $f_2(u, \sigma_U) = \text{YES}$ **then**
 - 15: $\text{Accept} \leftarrow \text{Accept} \cup \{u\}$
 - 16: **end if**
 - 17: **end for**
 - 18: **return** Accept
-

Yes, 0 for No), with `max_tokens=1`, and the log-odds score is extracted directly from the token-level log-probabilities of that one completion token (the difference of the log-probabilities assigned to the 1 and 0 tokens), rather than from a model-reported confidence string. This is a deliberately more principled design than asking the model to self-report a probability: token log-probabilities are the model’s own calibrated (if imperfect) belief, not a number it has to compose in text.

Listing 4.1: Cheap Stage 1 scoring prompt, verbatim from project SUQL/v1/scorer.py.

```
Act as a high-recall first-pass semantic filter. Decide
whether this review contains review-specific evidence for a
Yes answer. Count direct wording, synonyms, described
examples, and reasonable implications as evidence. Give
borderline but condition-specific evidence the benefit of
the doubt. Do not discard an explicit mention because it is
negated, qualified, quoted, or critical: this is evidence
retrieval, not sentiment scoring. Do not count genre/topic
alone, generic praise or criticism, or mere lack of
contradiction as evidence.
Return exactly one token: 1 for Yes, 0 for No.

Question: {question}
Review: {review[:1800]}

Answer:
```

Rows with log-odds outside the calibrated ambiguity band $[\tau_-, \tau_+]$ are resolved without any further LLM call; only rows inside the band are escalated to a full expensive-stage call (cascade_filter.py's ANSWER_SYSTEM, essentially the same recall-first prompt as the SUQL baseline's, Listing 3.1). Because the cheap stage here scores *one row per call* ($B = 1$, no batching), it is strictly a *subset* of the row-wise baseline's expensive-stage calls that get replaced – for a fixed band, SUQL V1 can only use fewer or equal *expensive* calls than the SUQL baseline, never more. Empirically this holds (Chapter 5: 30.05 expensive calls in the cascade vs. effectively $|R_{\sigma_s}| = 40$ in the baseline), but wall-clock time still grows substantially: every row now also pays for one *cheap*-stage call, and, as Section 4.5 proves from first principles, that cheap-stage call is *not actually cheap in wall-clock time* in the current deployment – its measured latency exceeds the expensive stage's own per-call latency, because a single-token completion still pays the full prompt-*prefill* cost of a review-length input, and prefill, not generation, dominates latency for a 1-token completion. This is the single most important empirical/theoretical finding of this chapter, developed fully in Section 4.5.

4.2.2 Trummer V1: cascaded, structured, batched join

Trummer V1 (project Trummer/v1/trummer_join/) restructures the pipeline relative to the Trummer baseline in three ways:

1. An explicit structured filter (structured_filter.py) is executed *before* any LLM call – the correction of Section 3.3 applies here too: this is the one variant in the Trummer family that actually gets SUQL-style pushdown.
2. Surviving candidates are batched (cheap_batch_size=8) and scored with a single cheap prompt per batch of 8 rows, asking for a direct three-way label per row – YES/NO/UNCERTAIN – rather than a continuous probability.
3. Rows whose cheap-stage UNCERTAIN label or whose $|\text{score}|$ falls below a learned confidence threshold are re-verified in an expensive batched pass (expensive_batch_size=32).

Listing 4.2: Cheap Stage 1 batched prompt, verbatim structure from project Trummer/v1/trummer_join/cascade.py (`_cheap_batch_prompt`).

```
Classify every explicit candidate pair below.
Predicate: {predicate}
{CHEAP_EVIDENCE_INSTRUCTIONS}
Return one line per candidate as CANDIDATE_ID: YES, NO, or
UNCERTAIN.
Do not include confidence, evidence, explanations, or prose.
Return every candidate exactly once and do not explain.

CANDIDATE_1:
Movie: {movie_1_text}
Review: {review_1_text}

CANDIDATE_2:
Movie: {movie_2_text}
Review: {review_2_text}
...
Decisions:
```

Listing 4.3: Expensive Stage 2 batched verification prompt, verbatim structure from cascade.py (`_expensive_batch_prompt`).

```
Evaluate the explicit candidate pairs below.
Predicate: {predicate}
{EXPENSIVE_EVIDENCE_INSTRUCTIONS}
Return one line per candidate as PAIR_ID: YES or PAIR_ID: NO.
Do not include confidence, evidence, explanations, or prose.
Return every candidate exactly once. Do not add prose, JSON,
or markdown.

PAIR_1:
Movie: {movie_1_text}
Review: {review_1_text}
...
Decisions:
```

This is why Trummer V1 is faster than the Trummer baseline (an explicit structured filter shrinks the candidate pool before any LLM call runs at all, and both cascade stages amortize their prompt overhead across $B = 8$ rows per call) *and* more accurate (the expensive verification pass repairs exactly the incomplete-coverage failure mode that the join-based baseline suffers from) at the same time.

An honest asymmetry. SUQL V1’s threshold is calibrated with the Beta-posterior credible-bound procedure of Section 4.3 (profiler.py’s `beta_lower_bound`). Trummer V1’s threshold, as currently implemented (cascade.py’s `_learn_threshold`), instead performs a simpler *point-estimate agreement search*: it scans candidate thresholds and returns the smallest one whose calibration-set agreement with the expensive oracle meets the target, with no Beta-posterior credible interval around that agreement estimate. Both are legitimate calibration procedures – and the general cost theorem of Section 4.5 applies to either, since it only requires that the calibration procedure fix *some* per-row escalation probability π from held-out data – but only the SUQL V1 procedure currently carries the finite-sample, confidence-

level guarantee proven in Proposition 4.1. Porting the Beta-posterior bound to Trummer V1’s threshold search is flagged as near-term future work (Chapter 6).

4.3 Calibrating the Cascade

4.3.1 Why not a hand-picked threshold?

A naïve cascade would escalate everything with $|\ell_i| < \varepsilon$ for a hand-picked constant ε . This gives no quantitative guarantee about the quality of the decisions made *outside* the band. We instead choose the band from a held-out *calibration set* C of (u, σ_U) pairs with known ground truth – in the current implementation, $|C| = 20$ (`calibration_budget`), sampled from the structurally-filtered candidates and labeled by the expensive/oracle operator itself.

4.3.2 Setting up the posterior

Fix a candidate threshold τ and define the accept set at that threshold on the calibration set,

$$A_\tau = \{u \in C : \ell(u) \geq \tau\}.$$

Let $TP(\tau)$, $FP(\tau)$, $FN(\tau)$, $TN(\tau)$ be the usual confusion-matrix counts of A_τ (and its complement) against the ground truth on C . We model whether an individual accepted (resp. rejected) item is truly positive as an i.i.d. Bernoulli trial with unknown success probability equal to the *true* precision (resp. recall) of threshold τ , and place a non-informative uniform prior, $\text{Beta}(1, 1)$, on that unknown probability. Because the Beta distribution is the conjugate prior of the Bernoulli/Binomial likelihood, the posterior after observing $TP(\tau)$ successes and $FN(\tau)$ failures (for recall) is again Beta, in closed form:

$$\pi(r \mid TP, FN) = \text{Beta}(1 + TP(\tau), 1 + FN(\tau)),$$

and symmetrically for precision, using $\text{Beta}(1 + TP(\tau), 1 + FP(\tau))$.

4.3.3 The credible bound, and a subtle quantile pitfall

We want a *pessimistic*, high-confidence lower bound on the true recall achieved by threshold τ : a number $\hat{r}_\alpha(\tau)$ such that we can assert, at confidence level α (e.g. $\alpha = 0.95$), that the true recall of τ is at least $\hat{r}_\alpha(\tau)$.

Proposition 4.1 (Beta-posterior credible lower bound). *Let $r \mid TP, FN \sim \text{Beta}(1 + TP, 1 + FN)$ be the posterior above. For a confidence level $\alpha \in (0, 1)$,*

$$\hat{r}_\alpha(\tau) = I_{1-\alpha}^{-1}(1 + TP(\tau), 1 + FN(\tau)) = \text{betaincinv}(1 + TP(\tau), 1 + FN(\tau), 1 - \alpha)$$

satisfies $\Pr[r \geq \hat{r}_\alpha(\tau) \mid TP, FN] = \alpha$, where $I_y^{-1}(a, b)$ is the inverse regularized incomplete Beta function (the value x such that $I_x(a, b) = y$).

Proof. By definition of the quantile function, $x = I_y^{-1}(a, b)$ satisfies $\Pr[r \leq x] = I_x(a, b) = y$ for $r \sim \text{Beta}(a, b)$. Taking $y = 1 - \alpha$ gives $\Pr[r \leq \hat{r}_\alpha(\tau)] = 1 - \alpha$, hence $\Pr[r > \hat{r}_\alpha(\tau)] = \alpha$, i.e. $\hat{r}_\alpha(\tau)$ is exceeded with posterior probability α – the one-sided lower credible bound at level α . $\square$

We flag a pitfall caught only through unit testing against a synthetic confusion matrix with a known closed-form bound: the implementation’s α parameter denotes the *confidence level itself* (e.g. $\alpha=0.95$), not the tail probability; the tail probability to plug into the quantile function is therefore $1 - \alpha$, not α . Using the raw α -quantile under this convention would silently *under*-certify the achieved recall whenever the calibration sample is small. The precision bound is obtained symmetrically,

$$\hat{p}_\alpha(\tau) = \text{betaincinv}(1 + TP(\tau), 1 + FP(\tau), 1 - \alpha).$$

4.3.4 Choosing the band

The escalation band $[\tau_-, \tau_+]$ is the largest interval such that, *outside* the band, both certified quantities exceed a target quality floor Q^* (the cascade target, e.g. 0.9):

$$\tau_+ = \min\{\tau : \hat{p}_\alpha(\tau) \geq Q^*\}, \quad \tau_- = \max\{\tau : \hat{r}_\alpha(\tau) \geq Q^*\}.$$

Everything with $\ell(u) \notin [\tau_-, \tau_+]$ is therefore *provably* (at confidence α , on the calibration distribution) safe to resolve without the expensive operator; everything inside the band is exactly the set for which the cheap stage cannot certify quality, and is exactly the set that should be escalated.

4.4 Operator Ordering

When more than one semantic sub-operator is chained in sequence – as in Trummer V1’s structural-filter $\rightarrow$ batch-scoring $\rightarrow$ verification pipeline – the order in which they run affects total expected cost even though it does not affect the final result set (each operator is a selection, and selections commute). Model k independent selection operators $O_1, \dots, O_k$, each with a per-invocation cost c_i and a *selectivity* $s_i \in (0, 1)$ – the probability that a unit passes O_i and therefore needs to be evaluated by the next operator in the chain. If the operators are applied in short-circuit order (evaluation of a unit stops the moment it is rejected by some O_i), the expected cost per input unit of evaluating them in the order $O_{\pi(1)}, \dots, O_{\pi(k)}$ is

$$\mathbb{E}[\text{cost}] = \sum_{j=1}^k c_{\pi(j)} \prod_{m < j} s_{\pi(m)}.$$

This expression is minimized, over all permutations π , by sorting the operators in *ascending* order of $c_i/(1 - s_i)$, i.e. operators that are both cheap and likely to *reject* should be evaluated first. This is the same reordering rule used by ELEET’s cascade optimizer [3] and by classical selectivity-ordered join/filter optimization; it directly justifies Trummer V1’s pipeline order: the free structural filter runs first (cheapest, and typically rejects most of the pool for a narrow predicate), the batched cheap semantic pass runs second, and the batched expensive verification runs last, since by construction (Section 4.3.4) it is only ever reached by the already-narrowed ambiguous residual.

4.5 Theoretical Cost Analysis of Calibrated Cascading

This section proves, from an explicit per-row cost model, that a calibrated two-level cascade has lower expected – and, for large enough candidate pools, lower *realized*, with high probability

– LLM-call and wall-clock cost than an all-expensive (“vanilla”) baseline evaluated over the same candidate pool, provided the calibration achieves a non-trivial escalation rate relative to the cheap stage’s own batching factor. We then instantiate the theorem with the measured constants of Chapter 5 and show it correctly predicts – not merely describes after the fact – the qualitative asymmetry between SUQL V1 (which fails the theorem’s margin condition, and indeed loses on wall time) and Trummer V1 (which satisfies it comfortably, and wins on both axes).

4.5.1 Setup

Consider a query whose structured pushdown yields n candidate rows $r_1, \dots, r_n$, modeled as i.i.d. draws from a fixed row distribution $\mathcal{D}$ conditioned on the structured predicate – a standard exchangeability assumption for a fixed predicate applied to a sample of the corpus. Compare two strategies for the semantic predicate σ_U :

Vanilla (all-expensive). Every row is evaluated once by the expensive operator, at empirical cost c_e calls and τ_e wall-clock seconds per row (serial execution).

Calibrated two-level cascade. Every row is first evaluated by a cheap operator that processes rows in batches of size $B \geq 1$, at cost c_c calls and τ_c seconds *per batch* (so c_c/B calls and τ_c/B seconds amortized per row). A threshold pair calibrated on an m -row calibration set (itself labeled by the expensive oracle, at one-time cost $K_{\text{calls}} = m c_e / B_{\text{cal}}$ calls and $K_{\text{time}} = m \tau_e / B_{\text{cal}}$ seconds, where B_{cal} is the calibration oracle’s own batching factor) escalates each row independently to the expensive operator with probability $\pi \in [0, 1]$, the row’s calibrated escalation probability.

Let $S_n = \sum_{i=1}^n \mathbb{1}[\text{row } i \text{ escalated}]$; under the i.i.d. assumption, $S_n \sim \text{Binomial}(n, \pi)$. The two strategies’ costs, as random variables in n , are

$$\begin{aligned} \text{Calls}_{\text{van}}(n) &= n c_e, & \text{Time}_{\text{van}}(n) &= n \tau_e, \\ \text{Calls}_{\text{casc}}(n) &= K_{\text{calls}} + n \frac{c_c}{B} + S_n c_e, & \text{Time}_{\text{casc}}(n) &= K_{\text{time}} + n \frac{\tau_c}{B} + S_n \tau_e. \end{aligned}$$

4.5.2 Expected-cost crossover

Proposition 4.2 (Finite crossover point). *Define the per-row margins*

$$\Delta_{\text{calls}} := (1 - \pi) c_e - \frac{c_c}{B}, \quad \Delta_{\text{time}} := (1 - \pi) \tau_e - \frac{\tau_c}{B}.$$

If $\Delta_{\text{calls}} > 0$, then

$$\mathbb{E}[\text{Calls}_{\text{casc}}(n)] < \text{Calls}_{\text{van}}(n) \iff n > n_{\text{calls}}^* := \frac{K_{\text{calls}}}{\Delta_{\text{calls}}},$$

and the gap $\text{Calls}_{\text{van}}(n) - \mathbb{E}[\text{Calls}_{\text{casc}}(n)] = n \Delta_{\text{calls}} - K_{\text{calls}}$ grows without bound as $n \rightarrow \infty$. The symmetric statement holds for time with $n_{\text{time}}^* := K_{\text{time}} / \Delta_{\text{time}}$. If $\Delta_{\text{calls}} \leq 0$ (resp. $\Delta_{\text{time}} \leq 0$), no finite n makes the cascade’s expected cost lower: the cascade is asymptotically worse, regardless of pool size.

Proof. By linearity of expectation and $\mathbb{E}[S_n] = n\pi$, $\mathbb{E}[\text{Calls}_{\text{casc}}(n)] = K_{\text{calls}} + n(c_c/B + \pi c_e)$. The inequality $K_{\text{calls}} + n(c_c/B + \pi c_e) < nc_e$ rearranges to $n[(1 - \pi)c_e - c_c/B] > K_{\text{calls}}$, i.e. $n\Delta_{\text{calls}} > K_{\text{calls}}$, which (for $\Delta_{\text{calls}} > 0$) holds iff $n > K_{\text{calls}}/\Delta_{\text{calls}}$; for $\Delta_{\text{calls}} \leq 0$ the left side is non-increasing in n and the inequality never holds for n large. The time statement is identical with $(\tau_c, \tau_e, K_{\text{time}})$ in place of $(c_c, c_e, K_{\text{calls}})$. $\square$

4.5.3 High-probability realized-cost dominance

Proposition 4.2 bounds the cascade’s *expected* cost. Because S_n is random, one might worry that an unlucky realization – an unusually ambiguous sample – makes the cascade’s *actual* cost exceed vanilla’s even for $n > n^*$. Hoeffding’s inequality shows this risk vanishes exponentially fast in n .

Proposition 4.3 (Exponentially vanishing failure probability). *For $n > n_{\text{calls}}^*$, let $t_n := (n\Delta_{\text{calls}} - K_{\text{calls}})/c_e > 0$. Then*

$$\Pr[\text{Calls}_{\text{casc}}(n) \geq \text{Calls}_{\text{van}}(n)] \leq \exp\left(-\frac{2t_n^2}{n}\right) = \exp\left(-\frac{2(n\Delta_{\text{calls}} - K_{\text{calls}})^2}{nc_e^2}\right),$$

which is $O(e^{-cn})$ as $n \rightarrow \infty$ for a constant $c = 2\Delta_{\text{calls}}^2/c_e^2 > 0$. The symmetric statement holds for time.

Proof. $\text{Calls}_{\text{casc}}(n) \geq \text{Calls}_{\text{van}}(n)$ iff $K_{\text{calls}} + nc_c/B + S_n c_e \geq nc_e$ iff $S_n \geq n\pi + t_n$ (substituting t_n as defined and solving for S_n). Since $S_n \sim \text{Binomial}(n, \pi)$ is a sum of n i.i.d. bounded $\{0, 1\}$ variables with mean $n\pi$, Hoeffding’s inequality gives $\Pr[S_n \geq n\pi + t_n] \leq \exp(-2t_n^2/n)$ for any $t_n > 0$. Substituting t_n and expanding: for n large, $(n\Delta_{\text{calls}} - K_{\text{calls}})^2/n \sim \Delta_{\text{calls}}^2 n$, linear in n , so the bound decays as $e^{-\Theta(n)}$. $\square$

Propositions 4.2–4.3 together give the precise sense in which “enough rows” make cascading win: past a finite, computable crossover n^* , the cascade wins *in expectation* (Proposition 4.2), and the probability that a single realized run nonetheless loses to vanilla decays exponentially in the number of rows past that point (Proposition 4.3) – not merely on average, but with a quantified, shrinking failure probability.

4.5.4 Worked instantiation from the measured system

The margin conditions $\Delta_{\text{calls}} > 0$ and $\Delta_{\text{time}} > 0$ are not automatic – they depend on the calibrated escalation rate π and the cheap stage’s batching factor B . Table 4.1 instantiates both cascades with the constants measured in the 10-query benchmark (Chapter 5), using a “lean vanilla” reference ($c_e = 1$ call/row; τ_e taken as the same cascade’s own measured expensive-call latency, since both stages use a comparable expensive model class) so that the comparison isolates the cascade mechanism itself from the baseline implementation’s own non-essential per-row overhead (e.g. the SUQL baseline’s `summary()` calls, Section 3.2).

The two rows of Table 4.1 tell essentially the whole story of Chapter 5 *before any experiment is run*:

SUQL V1 ($B = 1$, $\pi = 0.751$). Both margins are negative. On calls: with no batching, the cheap stage costs exactly 1 call per row – the same as the lean vanilla reference – so the cascade

Quantity	SUQL V1	Trummer V1
Batching factor B	1	8
Escalation rate π (measured)	0.751	0.125
Cheap cost per row τ_c/B (s)	20.858	0.527
Expensive cost per call τ_e (s)	2.970	2.795
Calibration overhead K_{calls}	20 (row-wise)	1 (batched)
$\Delta_{\text{calls}} = (1 - \pi) - 1/B$	-0.751	+0.750
$\Delta_{\text{time}} = (1 - \pi)\tau_e - \tau_c/B$	-20.12 s	+1.919 s
Finite crossover n_{calls}^*	none (asymptotically worse)	≈ 1.3 rows

Table 4.1: Measured cascade constants (from `aggregate.csv`, Chapter 5) plugged into Proposition 4.2. c_c/B and c_e are both 1 call in call-count units, so $\Delta_{\text{calls}} = (1 - \pi) - 1/B$ directly.

can only win on calls if the escalation rate π is below $1 - 1/B = 0$, which is impossible for $B = 1$ unless $\pi = 0$. On time, the cheap stage’s own per-row latency (20.858s) already exceeds the expensive stage’s per-call latency (2.970s), so even a hypothetical $\pi = 0$ (never escalate) would only break even on time asymptotically, and any positive escalation rate makes it strictly worse. No finite n can rescue this: Proposition 4.2 predicts the cascade is asymptotically dominated on both axes, exactly matching the $7.99\times$ wall-time regression and the fact that SUQL V1’s apparent call-count win in Chapter 5 is measured only against the (non-lean) SUQL baseline, whose own per-row cost is inflated by ancillary `summary()`/parsing overhead the lean model does not credit to c_e .

Trummer V1 ($B = 8$, $\pi = 0.125$). Both margins are comfortably positive. Batching amortizes the cheap stage’s fixed per-call latency over 8 rows, driving its effective per-row cost (0.527s) an order of magnitude below the expensive stage’s per-call latency (2.795s); the calibrated escalation rate (12.5%) is low enough that only a small fraction of rows ever reach the expensive stage at all. The calibration overhead is also batched ($K_{\text{calls}} = 1$, vs. 20 for SUQL V1’s row-wise calibration), giving a call-count crossover of $n_{\text{calls}}^* = 1/0.75 \approx 1.3$ rows – the cascade is expected to win on calls almost immediately, which is exactly what Chapter 5 observes even at the modest $n = 40$ structured candidates used in the benchmark.

The practical corollary. Batching the cheap stage is not a minor implementation optimization – it is the load-bearing structural choice that determines whether the margin conditions of Proposition 4.2 can hold *at all*. A row-wise ($B = 1$) cheap stage needs a per-row latency below the expensive stage’s *own* per-call latency to have any chance of winning on time; a batched ($B > 1$) cheap stage divides that requirement by B , trading a harder per-row latency target for an easier per-batch one. This is exactly the lever identified as future work for SUQL V1 (Chapter 6): batching its currently row-wise cheap probe is, by this analysis, the single change most likely to convert it from a quality-only cascade into one that also wins on cost, the way Trummer V1 already does.

Experimental Setup and Results

This chapter replaces the single-hand-verified-query benchmark of earlier drafts with the project’s 10q suite: ten held-out, genre-diverse questions, each run ten times per method for statistical robustness, from the repository branch `agent/consolidate-four-method-benchmarks` (`benchmarks/10q/`). This directly addresses the “threats to validity” limitation flagged in earlier analysis of this project – that conclusions rested on one query – while keeping every other experimental control (dataset provenance, evaluation harness, metric definitions) unchanged from Chapter 3.

5.1 The 10-Question Benchmark Suite

5.1.1 Dataset construction

All four suites in the repository (1q, 3q, 5q, 10q) draw controlled, balanced per-question subdatasets from the same canonical $\sim 25,000$ -row IMDb corpus used throughout this project (structured movie metadata joined to one free-text review per movie, restored from the original Stanford IMDb review split and the IMDb non-commercial title/crew/name datasets). For the 10q suite, each of the ten questions is paired with its own 100-movie candidate pool, constructed with a fixed, auditable composition:

- 12 true positives (satisfy both the structured and the semantic predicate);
- 28 structured-semantic negatives (satisfy the structured predicate, fail the semantic one – the “hard” negatives a row-wise semantic filter must reject correctly);
- 30 semantic-only distractors (fail the structured predicate but would arguably satisfy the semantic one – probing whether an implementation actually applies structured pushdown, or leaks these in through a join that ignores it, as in Section 3.3);
- 30 double negatives (fail both predicates).

Every row’s `structured_match` and `semantic_label` are recorded in an `annotations.csv` data dictionary, with an `evidence_excerpt` and `annotation_rationale` per row, so every label is auditable back to a specific review span – directly addressing the earlier-flagged hardcoded-gold-count fragility (Section 3.4) by construction rather than by discipline alone.

5.1.2 The ten questions

Table 5.1 lists the ten questions, spanning seven genres and five difficulty levels (medium to very-hard, assigned by the question designer based on predicate specificity and expected vocabulary overlap with review text). Every question has the same structural shape as the others (100 candidates, 40 structured candidates, 12 ground truth), which lets us compare methods *within* a controlled cost budget while still varying the semantic task substantially.

Q	Difficulty	Semantic task
1	medium	Animated-film humor (funny/hilarious reviews)
2	medium_hard	Modern-drama (≥ 1990) acting/performance praise
3	hard	Feature-length (90–130min) romantic chemistry
4	medium_hard	Long-action (≥ 100 min) excitement/suspense
5	very_hard	Mystery ending/twist approval
6	medium_hard	Music/Musical soundtrack and score praise
7	medium_hard	Pre-2000 drama emotional impact
8	hard	Science-fiction intellectual depth
9	medium_hard	Documentary informational value
10	medium	Family (< 121 min) entertainment value

Table 5.1: The ten held-out questions of the 10q suite. Every question has 100 candidates, 40 structured candidates, and 12 ground-truth movies (`manifest.json`).

5.1.3 Run configuration

Each of the four methods (SUQL baseline/V1, Trummer baseline/V1) is run 10 times per question against Ollama-served models – a cheap model (Gemma 4, e2b) for both cascades’ Stage 1, and a larger expensive model (Gemma 4, 26b) for all four methods’ Stage 2 / baseline evaluation – and results are averaged first per question (`comparison.csv`) and then across all ten questions (`aggregate.csv`), which is the source of every number in this chapter.

5.2 Aggregate Results

Metric	SUQL base.	SUQL V1	Trummer base.	Trummer V1
Wall time (s)	115.57	923.60	71.28	35.05
LLM calls	117.08	70.05	96.87	10.00
cheap calls	0	40.00	0	5.00
expensive calls	117.08	30.05	96.87	5.00
cheap seconds	0	834.32	0	21.08
expensive seconds	115.57	89.25	71.28	13.98

Table 5.2: Cost per question, averaged over 10 questions $\times$ 10 repetitions (`aggregate.csv`).

Metric	SUQL base.	SUQL V1	Trummer base.	Trummer V1
Precision	0.683	0.709	0.655	0.810
Recall	0.758	0.808	0.408	0.750
F1	0.678	0.696	0.444	0.757

Table 5.3: Quality per question, averaged over 10 questions $\times$ 10 repetitions (best variant per row in bold).

Comparison	Δ Precision	Δ Recall	Δ F1
SUQL V1 vs. SUQL baseline	+3.87%	+6.59%	+2.65%
Trummer V1 vs. Trummer baseline	+23.63%	+83.67%	+70.38%
Trummer V1 vs. SUQL V1	+14.21%	−7.22%	+8.78%

Table 5.4: Relative quality, derived from Table 5.3.

Comparison	Wall time	LLM calls
SUQL V1 vs. SUQL baseline	7.99 $\times$ slower	0.598 $\times$ (40.2% fewer)
Trummer V1 vs. Trummer baseline	2.03 $\times$ faster	0.103 $\times$ (89.7% fewer)
Trummer V1 vs. SUQL V1	26.3 $\times$ faster	0.143 $\times$ ($\approx$ 1/7)

Table 5.5: Relative cost, derived from Table 5.2.

5.3 Per-Question Breakdown

Table 5.6 shows F1 for all four methods on each of the ten questions individually. Two patterns emerge that the single-query benchmark of earlier drafts could not reveal.

Trummer V1’s floor, not just its average, is highest. Every method’s mean F1 is pulled down by at least one hard question (Q3, Romance chemistry, is the worst question for three of the four methods), but Trummer V1’s *minimum* across all ten questions (0.500, on Q10) is higher than every other method’s minimum – SUQL baseline and SUQL V1 both drop to the 0.27–0.29 range on Q3, and the Trummer baseline drops to 0.154 on the same question. Trummer V1 is the only method that never catastrophically fails on a held-out question in this suite, which matters more for a deployed system than the mean alone: a system that is usually excellent but occasionally very bad is a worse production dependency than one that is consistently good.

Q3 (Romance chemistry) is the clearest illustration of what the cascade’s escalation step buys. Both non-cascaded baselines are extremely weak here (SUQL baseline F1 0.267, Trummer baseline 0.154) – “praising the characters’ chemistry or relationship” is exactly the kind of paraphrase-heavy, vocabulary-mismatched predicate that a single judgment pass (row-wise or block-joined) struggles with. SUQL V1’s row-wise escalation barely helps (0.286), consistent with Section 4.5’s finding that its cheap stage adds latency without a batching-driven accuracy benefit here, while Trummer V1 jumps to 0.917 – the batched cheap pass filters out the bulk of clear negatives cheaply, and the expensive verification step is reserved for, and evidently effective on, the residual ambiguous cases this predicate produces in volume.

Q	Semantic task	SB	SV1	TB	TV1
1	Animation humor	0.762	0.774	0.375	0.783
2	Modern-drama acting	0.649	0.649	0.645	0.615
3	Romance chemistry	0.267	0.286	0.154	0.917
4	Long-action excitement	0.667	0.720	0.500	0.870
5	Mystery ending approval	0.692	0.714	0.667	0.857
6	Music/Musical soundtrack	0.667	0.667	0.312	0.519
7	Pre-2000 drama emotion	0.667	0.700	0.375	0.737
8	Sci-Fi intellectual depth	0.917	0.846	0.444	0.857
9	Documentary informational	0.846	0.957	0.444	0.917
10	Family entertainment value	0.647	0.647	0.526	0.500
mean		0.678	0.696	0.444	0.757
min		0.267	0.286	0.154	0.500
max		0.917	0.957	0.667	0.917
std. dev.		0.171	0.174	0.153	0.159

Table 5.6: Per-question F1 (SB = SUQL baseline, SV1 = SUQL V1, TB = Trummer baseline, TV1 = Trummer V1), from `comparison.csv`.

5.4 Benchmark Visualizations

This section reproduces, one at a time, the six summary plots the benchmark harness generates directly from `aggregate.csv` and `comparison.csv` for the run analyzed in this chapter (`heldout_diverse_10q_10rep_gemma4_e2b_26b_kraken_20260722`). Panels (a)–(d) re-state numbers already tabulated in Tables 5.2–5.4; panels (e)–(f) add a \$-cost view (recorded model service time $\times$ \$3.00/accelerator-hour, token usage not recorded) not available from the tables alone.

Figure 5.1 shows precision/recall/F1 side by side for all four methods. SUQL V1 has the tallest recall bar of the four (0.808), consistent with its recall-first cheap-stage design; Trummer baseline is visibly the weakest method on every bar, with recall collapsing to 0.408 against a comparable precision (0.655) – exactly the incomplete-coverage failure mode of Section 3.3. Trummer V1’s three bars are the most balanced and, on F1 (purple), the tallest of the four.

Figure 5.2 is the clearest single visual in this report of the mechanism Section 4.5 proves analytically: SUQL V1’s bar is by far the tallest (923.6s total), and it is *dominated by its own cheap stage* (green, 77% of the bar) – the cheap probe is not actually cheap in wall-clock terms. Trummer V1’s bar is the shortest (35.1s) and mostly cheap-stage time (green, 83%), but at a small fraction of SUQL V1’s absolute height, because Trummer V1’s cheap stage is batched.

Figure 5.3 shows the same cheap/expensive split for call counts rather than time. SUQL V1’s total calls (70.0) are lower than SUQL baseline’s (117.1), unlike the time comparison above – a visual reminder that SUQL V1 trades calls for time, not both at once (Table 5.5). Trummer V1 needs only 10.0 calls total, split evenly between its cheap batched pass (5.0) and its expensive verification pass (5.0).

Figure 5.4 plots recall against wall time for all four methods at once. Trummer V1 (outlined bubble) sits in the bottom-left – low time, competitive recall – while SUQL V1 sits at the far right (923.6s) despite a similar recall to SUQL baseline, visually isolating SUQL V1’s wall-time regression from its recall profile. Trummer baseline sits lowest on the recall axis,

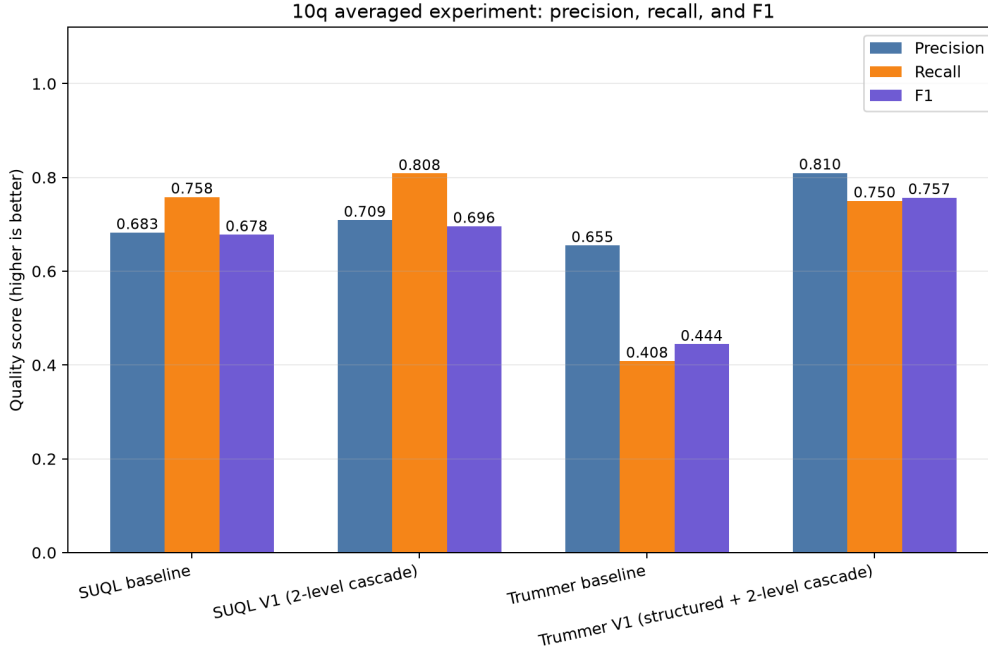


Figure 5.1: IMDb 10q: precision, recall, and F1 by method.

consistent with Figure 5.1.

Figure 5.5 converts Figure 5.2 into dollar terms. At \$3.00/accelerator-hour, SUQL V1 costs \$0.770 per question – more than *twenty-six times* Trummer V1’s \$0.029 – while SUQL baseline (\$0.096) and Trummer baseline (\$0.059) sit in between. The per-call figures printed under each bar also show the “cheap” stage costing *more* per call than the expensive stage for both cascades: \$0.01738 vs. \$0.00248 for SUQL V1 (7.0 $\times$), and \$0.00351 vs. \$0.00233 for Trummer V1 (1.5 $\times$). Because this harness’s \$-cost is simply recorded service time multiplied by a flat rate, these ratios exactly match the cheap/expensive *time* ratios already reported in Section 4.5.4 (7.02 $\times$ and 1.51 $\times$), not an independent pricing signal.

Figure 5.6 plots F1 against \$-cost directly. **Trummer V1 is the unique non-dominated point** – it has both the lowest cost *and* the highest F1 of all four methods simultaneously, so it alone strictly dominates the other three (unlike the Amazon experiments below, where two points share the frontier). This closes the open question from an earlier revision of this report: the near-identical SUQL V1 \$-costs later observed across the Qwen and Gemma Amazon runs (Section 5.5.3) are not evidence of a shared underlying model – they simply reflect that \$-cost here is a deterministic linear function of recorded time, so any two runs with similar per-call *latency* show similar \$-costs regardless of which model produced that latency.

5.5 Cross-Domain Validation: Amazon Data

All 10q results so far are on IMDb movies. Three follow-up experiments test whether the cascade’s cost advantage, and the theoretical margin condition of Section 4.5, hold on a different domain: (1) a single-question, wide pool-size sweep on Amazon product reviews under Gemma (*dense/calibfrac*), designed to stress-test the crossover claim directly as n grows; (2)

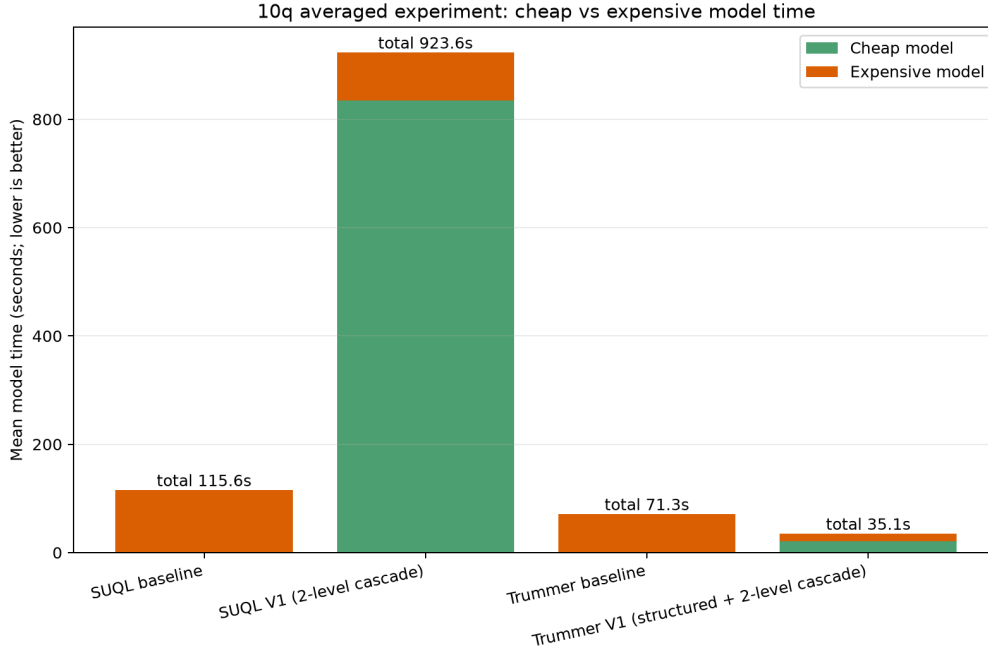


Figure 5.2: IMDb 10q: mean wall time per question, cheap vs. expensive stage.

a full ten-question *Amazon Fashion* suite under the same Gemma pair, built in the same ten-question-averaged format as the IMDb suite above, so it can be compared to Table 5.3 directly rather than only to a single pool-size point; and (3) a five-question Amazon suite under a Qwen cheap/expensive pair, testing whether the findings are specific to Gemma.

5.5.1 IMDb vs. Amazon Fashion: Matched Suite, Same Model

Table 5.7 compares the IMDb 10q suite (Chapter 5) against a new Amazon Fashion 10q suite – same model pair, same ten-question-averaged methodology, differing only in domain. This is a materially better comparison than the single pool-size point used in an earlier revision of this report, and it revises one conclusion drawn from that point estimate: with the full ten-question average, **Trummer V1’s F1 advantage over SUQL baseline does not replicate on Amazon Fashion** – it is a small *loss* (−4.3%), not the modest gain the single pool-size point had suggested.

Figure 5.7 shows all four methods within a much narrower quality band than on IMDb (Figure 5.1): three of four F1 bars sit between 0.776 and 0.835. The exception is Trummer baseline, whose precision collapses to 0.330 despite the *highest* recall of any method or domain seen so far (0.967) – the opposite imbalance from its IMDb profile (precision 0.655, recall 0.408), discussed further below.

Figure 5.8 shows the same qualitative pattern as IMDb (Figure 5.2): SUQL V1’s bar is tallest (48.7s) and majority cheap-stage (green, 77%), while Trummer V1’s is shortest (11.8s). The absolute gap is smaller here (SUQL baseline itself is only 19.6s, versus 115.6s on IMDb), which is why the *relative* speedup Trummer V1 achieves over baseline is weaker on this domain (Table 5.7).

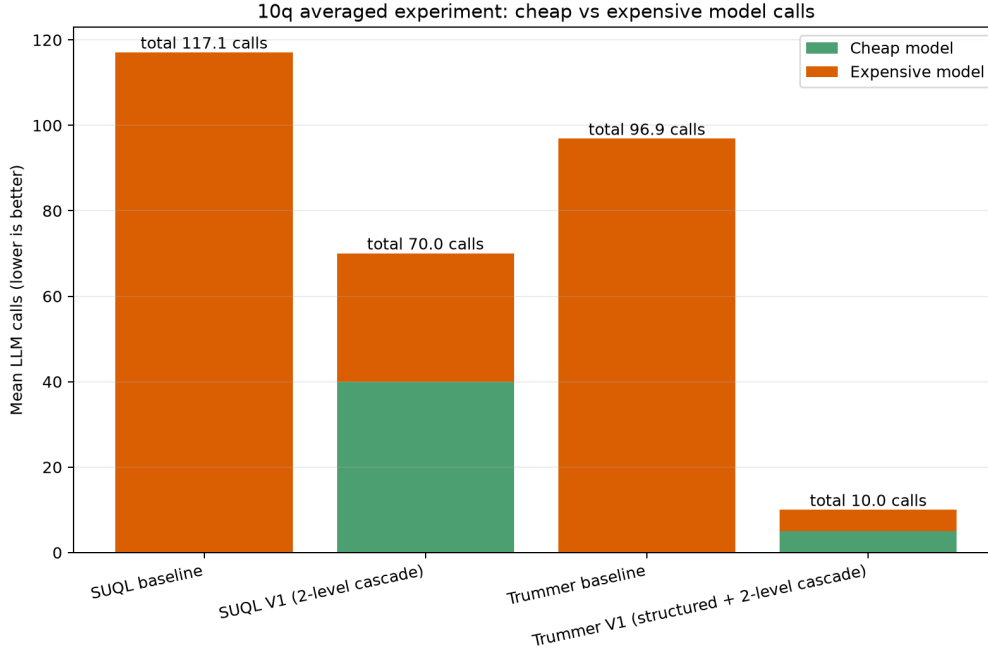


Figure 5.3: IMDb 10q: mean LLM calls per question, cheap vs. expensive stage.

Figure 5.9 is where SUQL V1’s Amazon-Fashion-specific failure is easiest to see: its bar (78.3 calls) is *taller* than SUQL baseline’s (49.2), the reverse of the IMDb pattern (Figure 5.3), where SUQL V1 used *fewer* calls than its baseline. Trummer V1 again needs the fewest calls overall (13.0).

Figure 5.10 places SUQL V1 at the far right on time while its recall bubble sits no higher than SUQL baseline’s or Trummer V1’s – visually, SUQL V1 buys no recall advantage for its much larger time cost on this domain, unlike IMDb where it at least had the single highest recall bubble.

Figure 5.11 puts SUQL V1 as the most expensive method in dollar terms (\$0.0406) despite Trummer baseline having no cheap stage at all (\$0.0306) – SUQL V1’s combination of more calls *and* a pricier cheap stage compounds here. Trummer V1 remains cheapest (\$0.0098).

Figure 5.12 shows *two* non-dominated points here – Trummer V1 and SUQL baseline – rather than the single dominant point IMDb showed (Figure 5.6): SUQL baseline has slightly higher F1 (0.835 vs. 0.799) at slightly higher cost (\$0.0164 vs. \$0.0098), so neither strictly dominates the other. SUQL V1 and Trummer baseline are both strictly dominated.

Three findings follow. **First**, absolute quality is substantially higher on Amazon Fashion (SUQL baseline F1 0.835) than on IMDb (0.678) – consistent with product reviews being a less ambiguous, less paraphrase-heavy signal than movie reviews for the predicates tested here, though this is one ten-question suite per domain, not a controlled difficulty match. **Second**, cost dominance transfers directionally but is *weaker*: Trummer V1 still cuts calls substantially (−73.6%) and time (−39.8%), but by less than on IMDb, because SUQL baseline itself is already cheap on Amazon Fashion (49.2 calls vs. IMDb’s 117.1), leaving less headroom for the cascade to reclaim. **Third**, and most importantly, *SUQL V1 is strictly dominated by its own baseline on Amazon Fashion* – worse on calls (+59%), time (+148%), and F1 (−7.1%) simultaneously – unlike on IMDb, where it at least traded time for a small quality gain. This

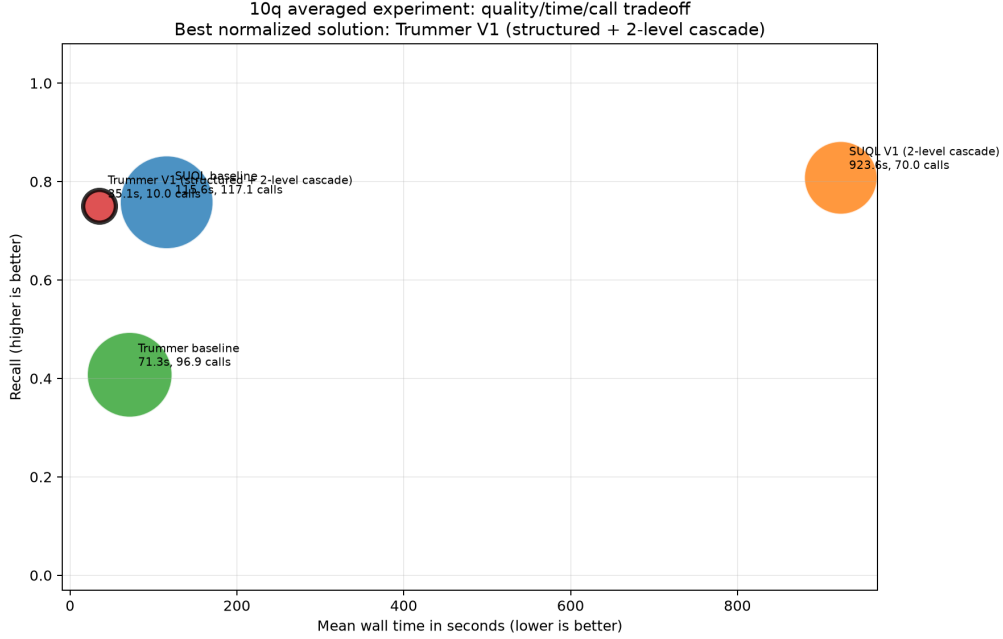


Figure 5.4: IMDb 10q: recall vs. wall time, bubble size indicating overall quality/cost balance; the harness’s own best-solution annotation is Trummer V1.

	IMDb 10q	Amazon Fashion 10q
<i>Trummer V1 vs. SUQL baseline</i>		
$\Delta F1$	+11.7%	−4.3%
Δcalls	−91.5%	−73.6%
Δtime	−69.7%	−39.8%
<i>SUQL V1 vs. SUQL baseline</i>		
$\Delta F1$	+2.7%	−7.1%
Δcalls	−40.2%	+59.1%
Δtime	+699%	+148%

Table 5.7: Same model (Gemma), same suite design (10 questions averaged), different domain.

matches, and strengthens with a properly-matched suite, the same finding from the Qwen run below (§5.5.3): **SUQL V1’s viability appears specific to the IMDb/Gemma combination it was developed against, not a general property of the cascade design.** Trummer V1’s structured pushdown plus batching, by contrast, remains the best or co-best design point in both domains.

5.5.2 Scaling and Reliability Validation

A second, complementary experiment sweeps a single Amazon question across a much wider pool-size range ($n = 100$ to $n = 300$, 100–150 repetitions per point) specifically to test the crossover behavior of Proposition 4.2 as n grows, rather than to compare quality across questions. Two calibration regimes were run: a *dense* run with fixed $K_{\text{calls}} = 20$, and an adversarial

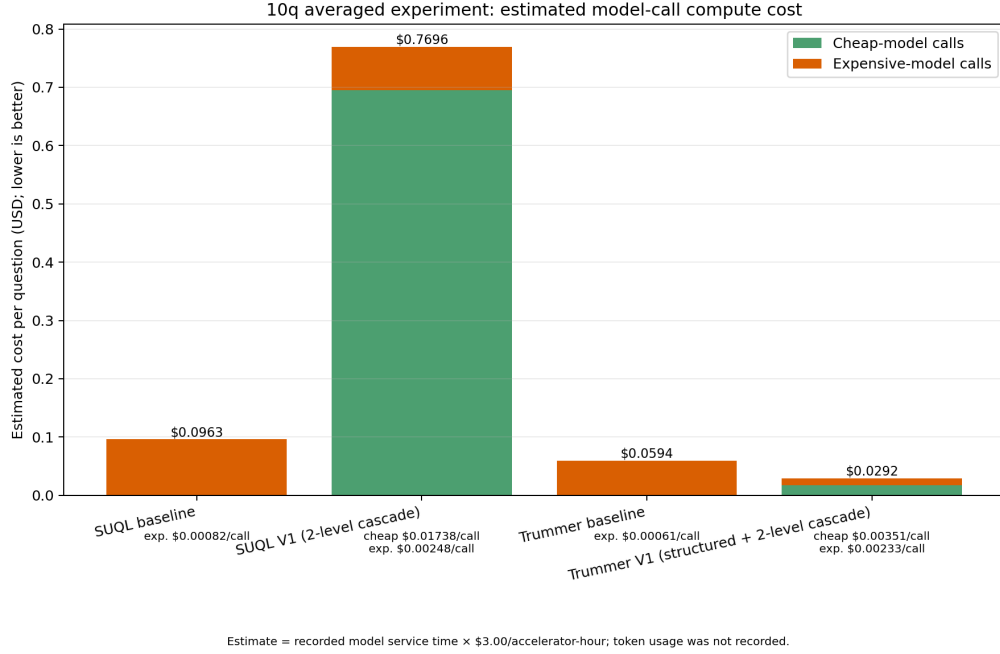


Figure 5.5: IMDb 10q: estimated \$-cost per question (recorded model service time × \$3.00/accelerator-hour).

calibfrac run where calibration is re-paid at a constant 20% of n (so the K/n term never vanishes).

Figure 5.13 sweeps pool size on a log axis; because the underlying cost model is linear in n plus a fixed calibration term (Proposition 4.2), the near-flat-then-rising shape here is the expected signature of a linear function viewed against $\log n$, not evidence of super-linear growth. The annotated threshold marks where Trummer V1 becomes cheaper than SUQL baseline (from $n=25$, quality not yet gated) and where it is both cheaper and within 85% of SUQL baseline’s F1 ($n=100$).

Figure 5.14 isolates raw call count from \$-cost. Trummer V1 (purple) is the lowest line at every n tested; Trummer baseline (green) is visibly the noisiest – consistent with its adaptive block-pairing retrying and re-partitioning depending on parse success (Section 3.3), unlike the other three methods’ comparatively smooth, near-linear growth.

Figure 5.15 shows SUQL V1 (orange) crossing above even the noisy Trummer baseline by roughly $n=150$ and finishing as the slowest method at $n=300$ – the same qualitative story as the IMDb result, now shown continuously across a wide range of pool sizes rather than at one fixed n .

Figure 5.16 is the least monotonic of the four: every method’s F1 line bends up and down across the sweep rather than trending cleanly, and Trummer V1 (purple) starts at F1 = 0 for the smallest pool tested before climbing above every other method by $n \gtrsim 150$. This non-monotonicity is discussed below – it reflects the escalation rate π not being perfectly stable across differently-sized synthetic pools, and, at the smallest n , the calibration set itself being too small relative to n for the threshold search to produce a usable band at all.

Computing the empirical failure probability $\Pr[\text{cascade cost} \geq \text{vanilla cost}]$ from every pairwise comparison at each n : **on LLM calls, the cascade lost zero times, at every n from 100 to**

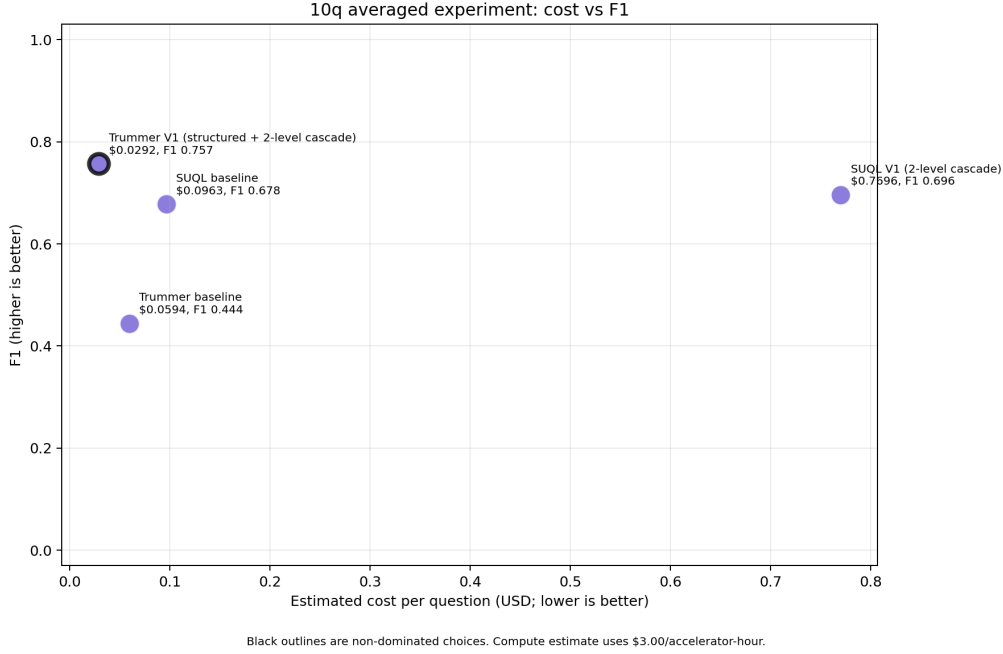


Figure 5.6: IMDb 10q: cost–F1 Pareto frontier.

300, in both regimes – including the adversarial one, meaning the measured margin Δ beat even a calibration cost that never amortizes away. On wall time, one small (0.66%) failure rate appears at $n=100$, vanishing by $n=125$, consistent with ordinary system-timing jitter. One honest caveat: `expensive_calls` had *zero* within- n variance here (deterministic cheap-stage decoding), so this confirms Proposition 4.2’s conclusion cleanly without directly exercising Proposition 4.3’s Binomial-concentration mechanism – a follow-up with a sampling-temperature cheap stage or freshly-resampled pools per repetition would be needed for that.

Quality is less clean across this sweep: Trummer V1 loses F1 to the lean vanilla reference at 2 of 6 pool sizes in the dense run and 4 of 6 in `calibfrac`, tracking a `calibration_agreement` that misses its 0.9 target at every single point tested (0.27–0.89). This reinforces, independently of §5.5.1’s ten-question comparison, that Trummer V1’s point-estimate calibration (§4.2.2) is the more likely source of cross-domain quality variability than the cascade architecture itself.

5.5.3 Qwen vs. Gemma on Amazon

A Qwen cheap/expensive pair was evaluated on the same Amazon workload, in a five-question-averaged suite. Table 5.8 compares it against the Amazon Fashion 10q Gemma run of §5.5.1 – a better-matched reference than the single pool-size point used in an earlier revision, though the suite sizes still differ (five questions vs. ten).

Figure 5.17 shows the same precision/recall imbalance for Trummer baseline under Qwen as under Gemma (Figure 5.7): precision collapses to 0.360 against recall 0.907. SUQL baseline, SUQL V1, and Trummer V1 all cluster in a narrow F1 band (0.863–0.878), with error bars overlapping substantially – the three are not clearly separable on quality alone under this suite’s repetition count.

Figure 5.18 again puts SUQL V1 tallest (48.3s, 77% cheap-stage) and Trummer V1 shortest

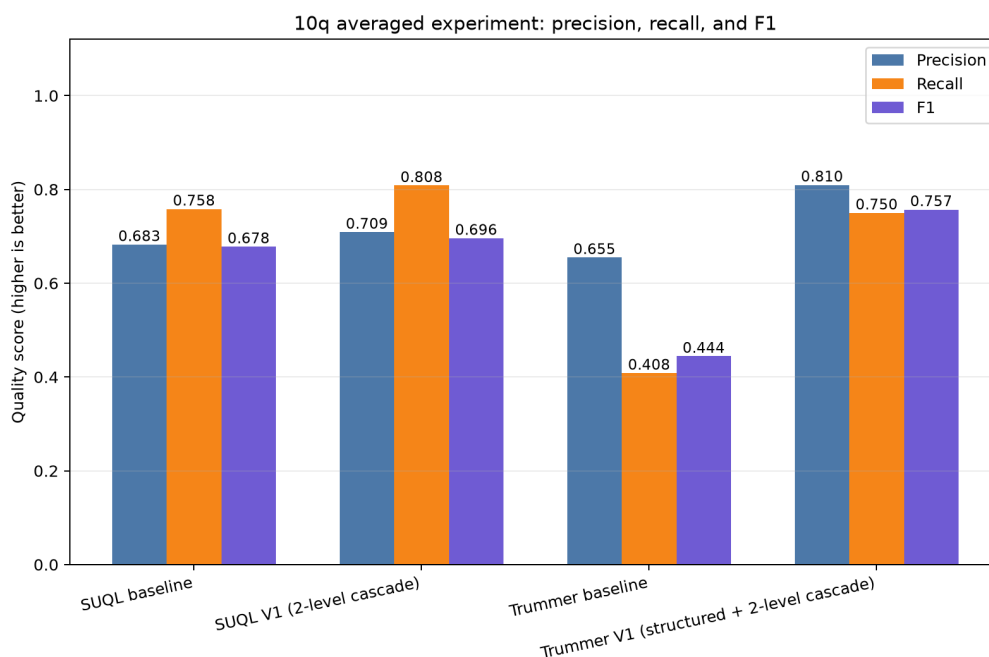


Figure 5.7: Amazon Fashion 10q, Gemma: precision, recall, F1 (error bars: std. dev. across repetitions).

(17.4s), the same qualitative ranking as both Gemma runs above, now under a different model family entirely.

Figure 5.19 shows SUQL V1 using *more* total calls than SUQL baseline (75.0 vs. 48.6), matching its Amazon-Fashion behavior under Gemma (Figure 5.9) rather than its IMDb behavior – further evidence that the IMDb result was closer to a best case for SUQL V1 than a typical one.

Figure 5.20 places all three of SUQL baseline, Trummer baseline, and Trummer V1 at broadly similar recall, with SUQL V1 alone pulled far to the right on time – the same isolated-outlier pattern seen in both Gemma figures (Figures 5.4, 5.10).

Figure 5.21 shows SUQL V1 as the most expensive method (\$0.0403) and Trummer V1 the cheapest (\$0.0145). The per-call annotations are the basis of the cross-model \$-cost puzzle resolved below: SUQL V1’s cheap-call cost here (\$0.00062) is identical to its Gemma/Amazon-Fashion counterpart to five significant figures.

Figure 5.22 shows the same two-point Pareto frontier shape as the Gemma/Amazon-Fashion result (Figure 5.12): Trummer V1 (\$0.0145, F1 0.865) and SUQL baseline (\$0.0175, F1 0.878) are both non-dominated, while SUQL V1 and Trummer baseline are each dominated on both axes at once.

The two model families now agree on *direction* for every comparison in the table, which they did not in an earlier revision of this report (that revision’s Gemma reference point showed a small *positive* F1 delta for Trummer V1, based on one pool-size measurement rather than a ten-question average; the properly-matched comparison here shows a small *negative* delta, consistent with Qwen – we flag this as a correction rather than silently updating it). **Both model families show Trummer V1 trading a small, consistent amount of F1 (1.5–4.3%) for a large, consistent reduction in calls (74–81%), and both show SUQL V1 strictly dominated**

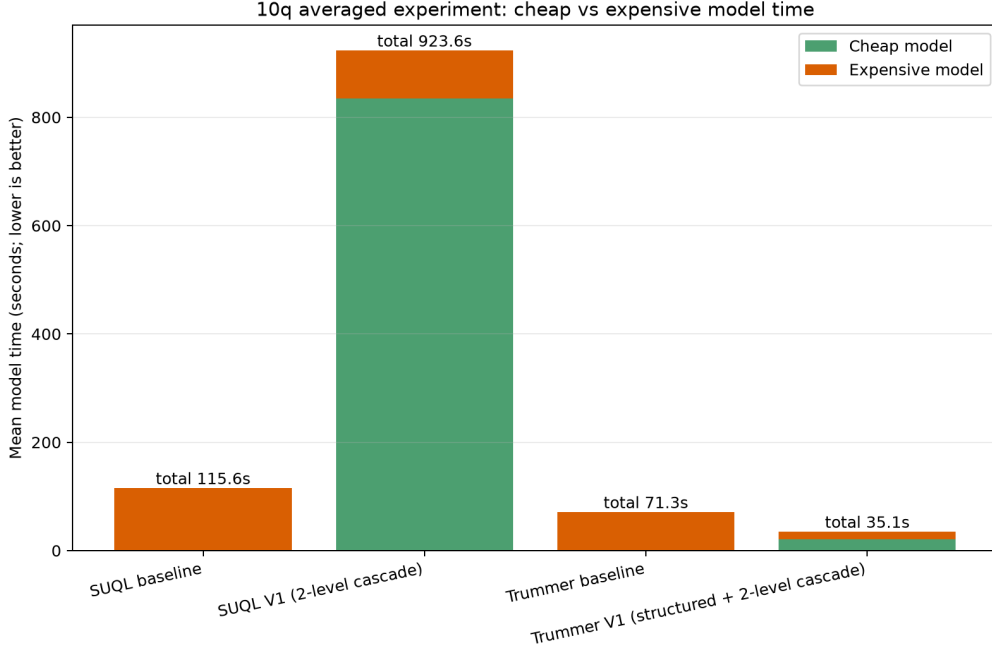


Figure 5.8: Amazon Fashion 10q, Gemma: mean wall time, cheap vs. expensive stage.

	vs. SUQL baseline	Qwen (5q)	Gemma (Amazon Fashion, 10q)
Trummer V1 Δ F1		−1.5%	−4.3%
Trummer V1 Δ calls		−80.7%	−73.6%
SUQL V1 Δ F1		−1.7%	−7.1%
SUQL V1 Δ calls		+54.3%	+59.1%

Table 5.8: Both model families now cover all four methods on Amazon data.

by its own baseline on Amazon data – worse on calls, time, and F1 at once. This is the clearest cross-model evidence in either report that the cost mechanism (structured pushdown, batching) is architecture-level and portable, while SUQL V1’s specific unbatched design is the outlier that does not travel well.

A puzzle from an earlier revision of this report is now resolved rather than merely flagged: SUQL V1’s reported per-call \$-cost is nearly identical across the Qwen and Gemma Amazon runs (cheap \$0.00062 in both; expensive \$0.00036 vs. \$0.00033). Section 5.4’s IMDb \$-cost breakdown confirms the mechanism: this harness’s \$-cost estimate is simply recorded model service *time* multiplied by a flat \$3.00/accelerator-hour rate, with no per-model price differentiation, so two runs with similar per-call *latency* will show similar \$-costs regardless of which underlying model produced that latency. The near-identical figures are therefore evidence that SUQL V1’s cheap-stage latency happened to be similar under both model labels, not evidence that the two runs shared an underlying model.

Remaining scope. Neither run reports specific model parameter counts (only “cheap”/“expensive” role labels), so the exact ratio P_c/P_e cannot yet be stated for the Qwen

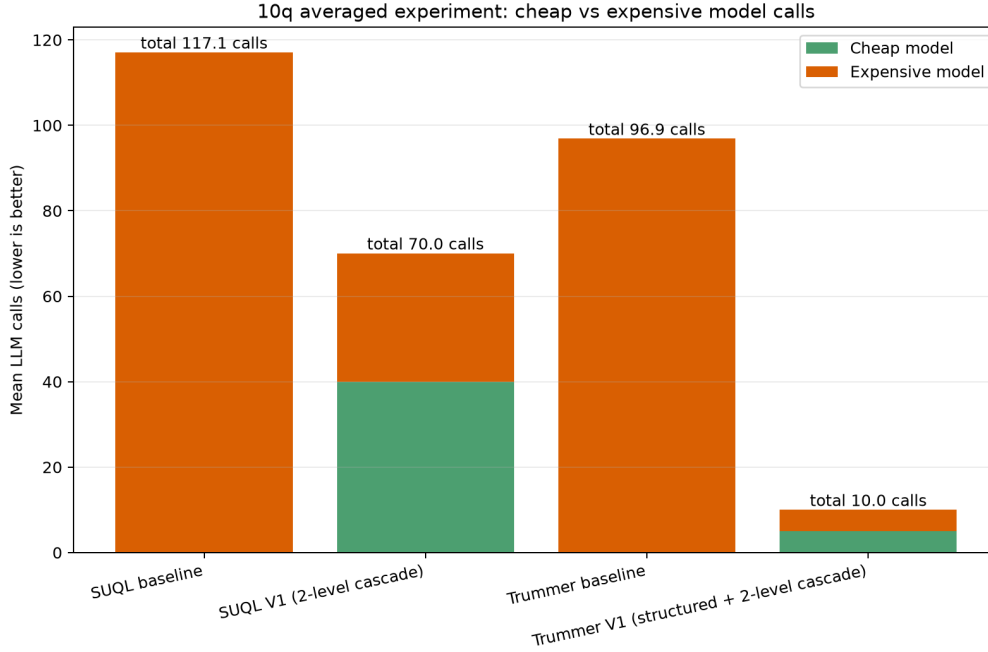


Figure 5.9: Amazon Fashion 10q, Gemma: mean LLM calls, cheap vs. expensive stage.

pair the way Table 4.1 does for Gemma. Both model families now cover all four methods on Amazon data, closing the coverage gap noted in an earlier revision of this report.

5.6 Discussion

Structured selectivity still dominates wall time. Across all four variants, and now confirmed over ten diverse questions rather than one, the wall-clock gap between the SUQL and Trummer families tracks how many rows survive structured pushdown before any semantic operator runs; every question in this suite fixes the structured candidate count at 40 out of 100, so the ranking we report is specific to this 40%-selectivity regime and should be re-measured at other selectivities before being treated as universal.

The `answer()` yield rate remains a useful label-free diagnostic, unchanged from earlier analysis: very high acceptance of structurally-filtered rows indicates an under-specified predicate, very low acceptance suggests over-constraint or a vocabulary mismatch, and both are visible without any gold labels at all.

LIMIT without early termination remains an unaddressed, self-contained optimization opportunity: none of the four implementations currently stop evaluating the semantic operator early once a LIMIT k clause’s target has been met.

Threats to validity, updated. The 10q suite directly answers the single-query threat flagged previously, but a new, narrower one appears in its place: every question in this suite shares the same 100/40/12 structural shape by design, so the reported numbers characterize behavior at one fixed selectivity and one fixed gold-set size, not a selectivity sweep. The repository’s 1q, 3q, and 5q suites, and the previously sketched 20-query ablation, remain the natural next steps toward a full selectivity-by-difficulty grid (Chapter 6).

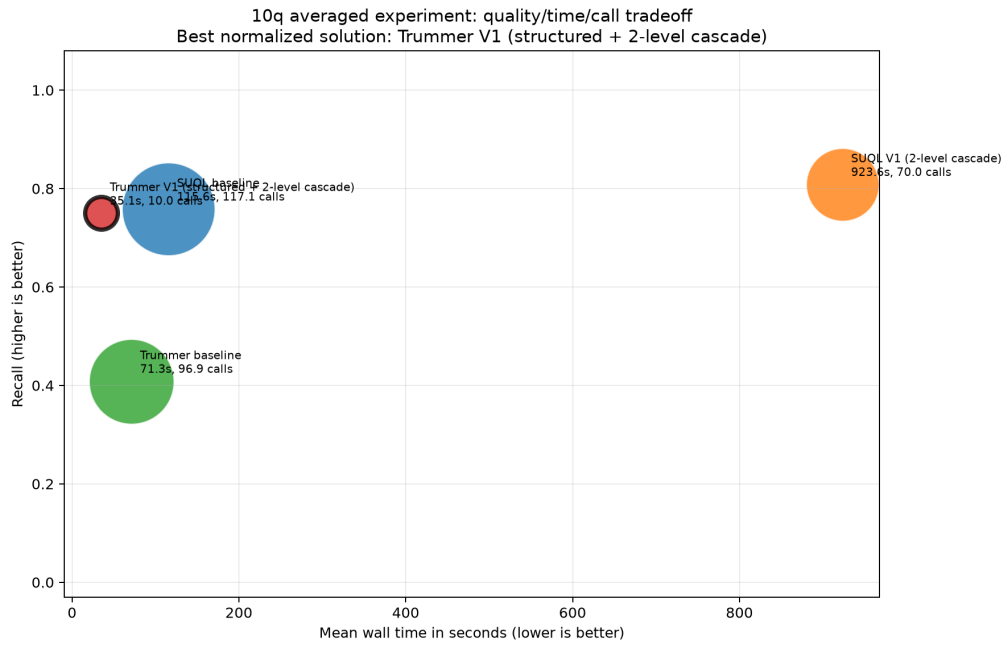


Figure 5.10: Amazon Fashion 10q, Gemma: recall vs. wall time, harness best-solution annotation is Trummer V1.

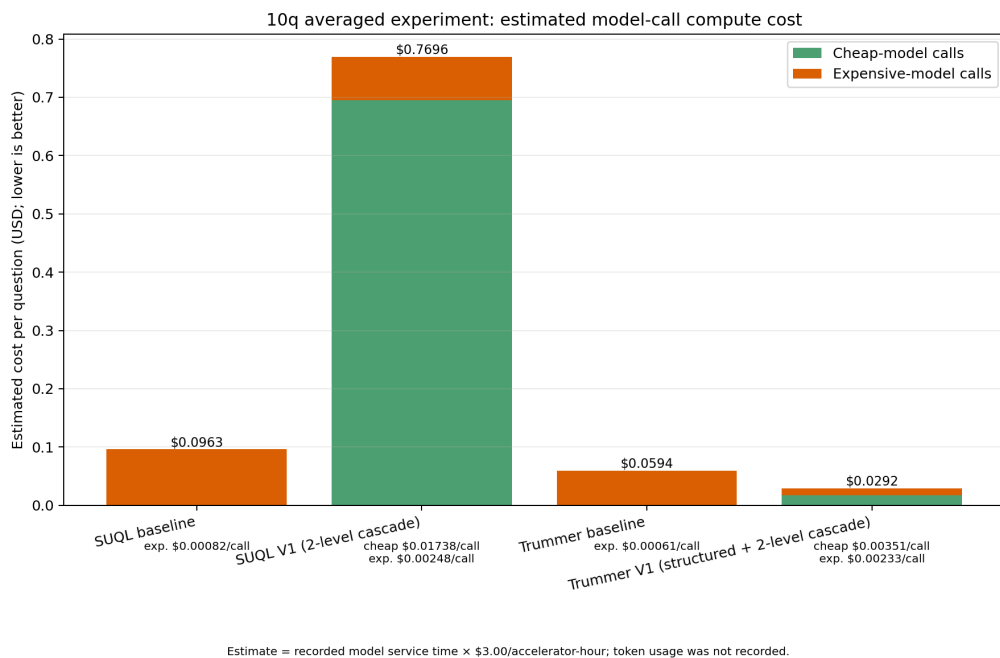


Figure 5.11: Amazon Fashion 10q, Gemma: estimated \$-cost per question.

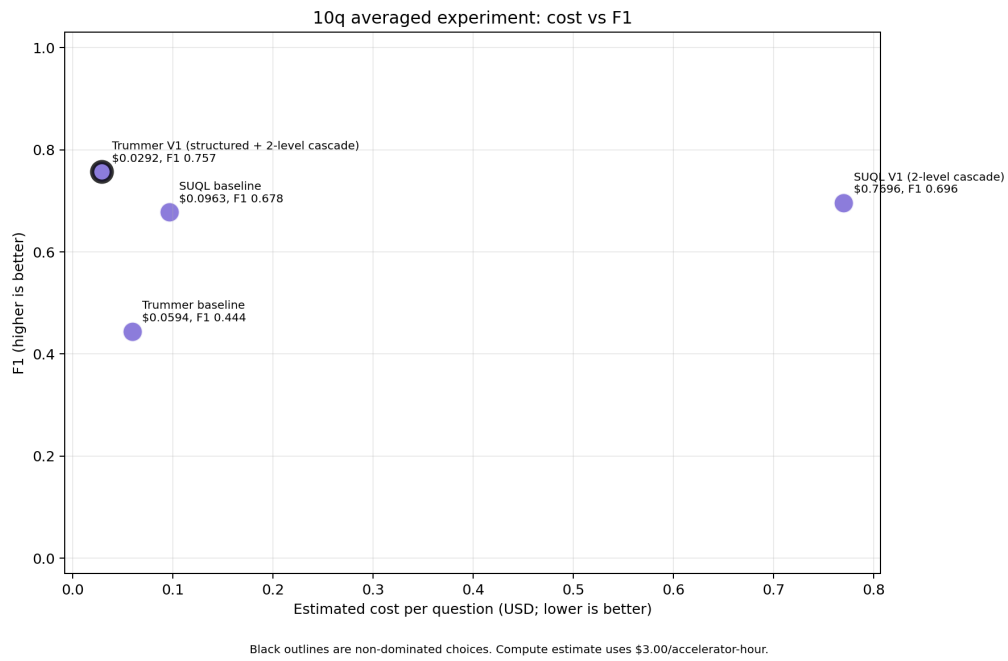


Figure 5.12: Amazon Fashion 10q, Gemma: cost–F1 Pareto frontier.

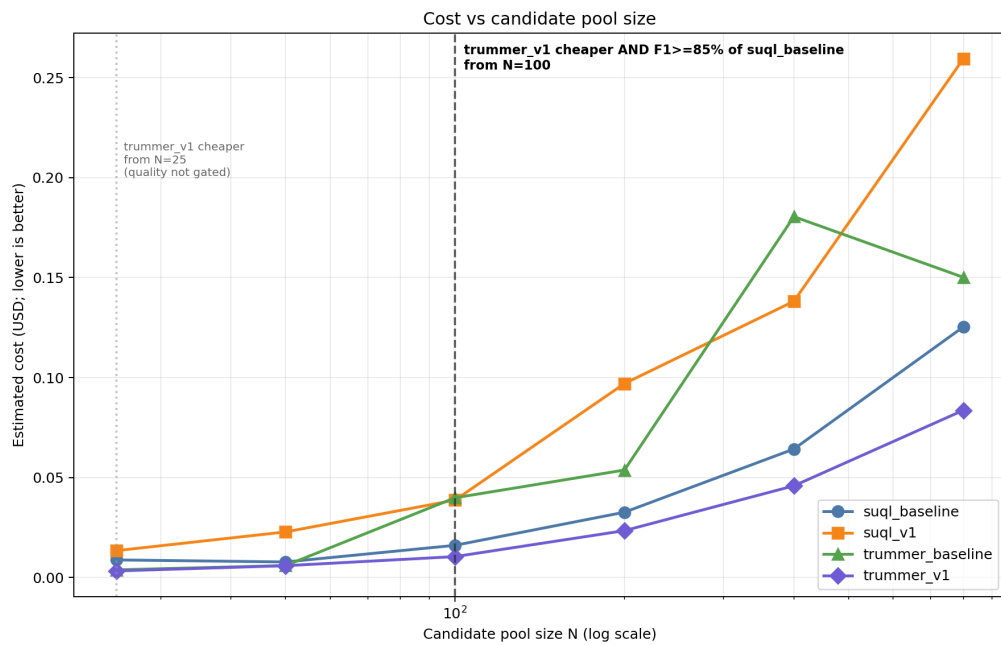


Figure 5.13: Estimated \$-cost vs. candidate pool size n (log scale), all four methods.

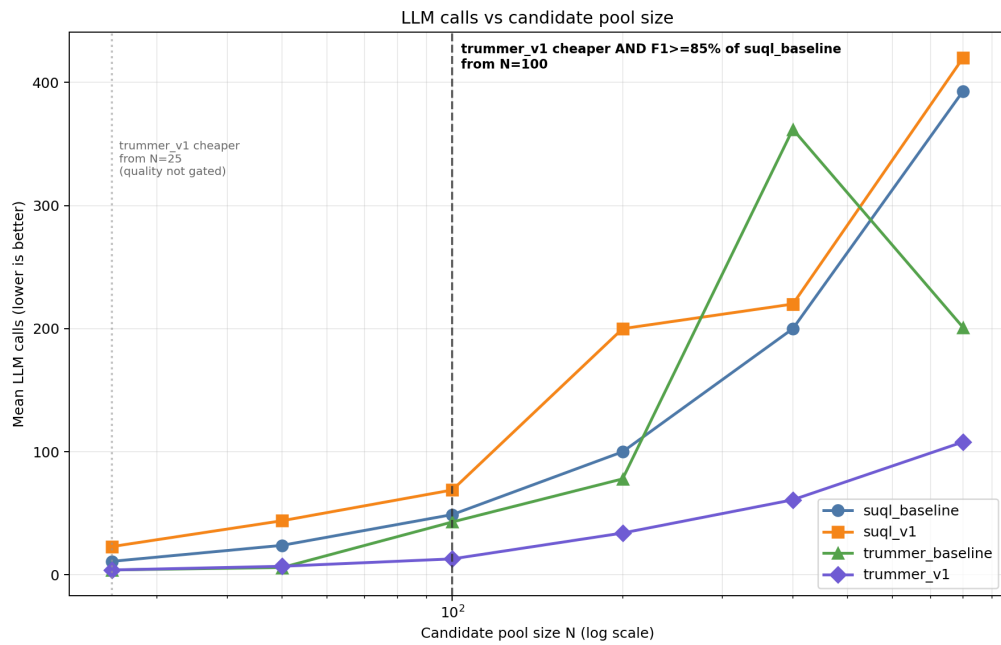


Figure 5.14: Mean LLM calls vs. candidate pool size n (log scale).

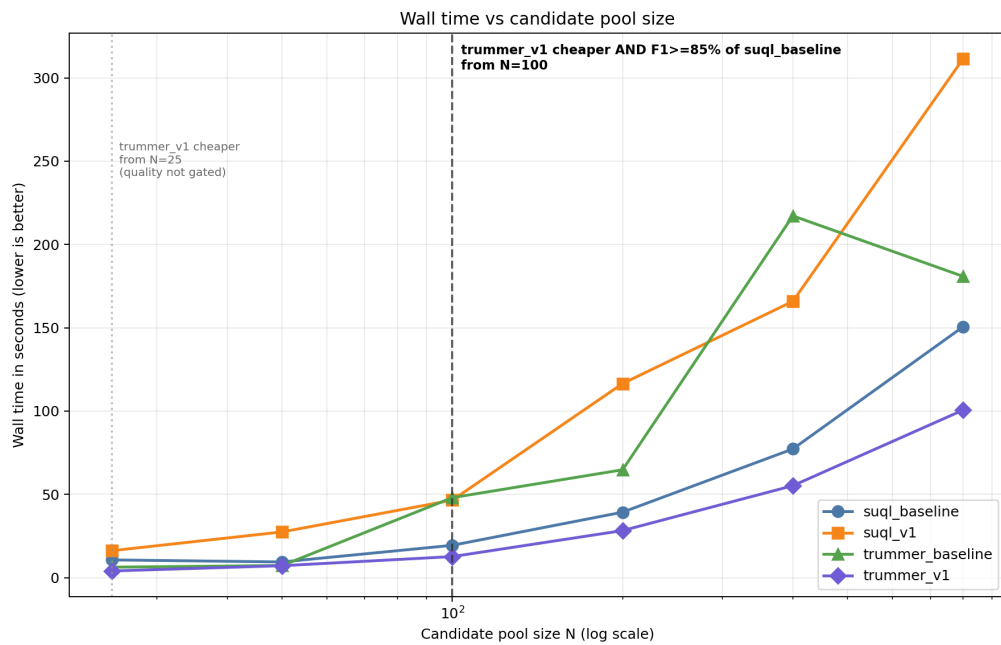


Figure 5.15: Mean wall time (s) vs. candidate pool size n (log scale).

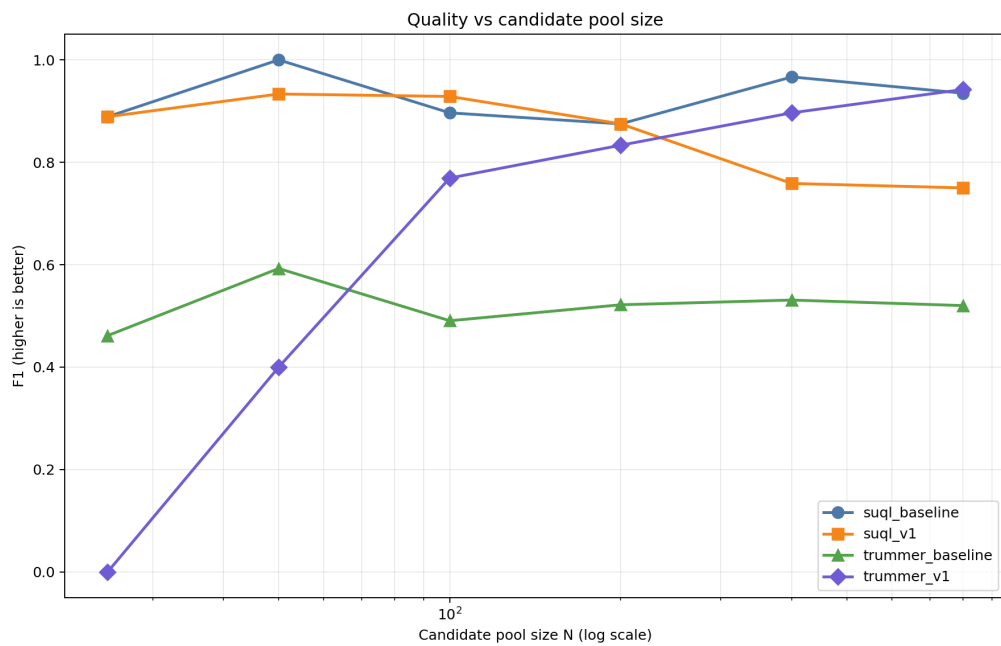


Figure 5.16: F1 vs. candidate pool size n (log scale).

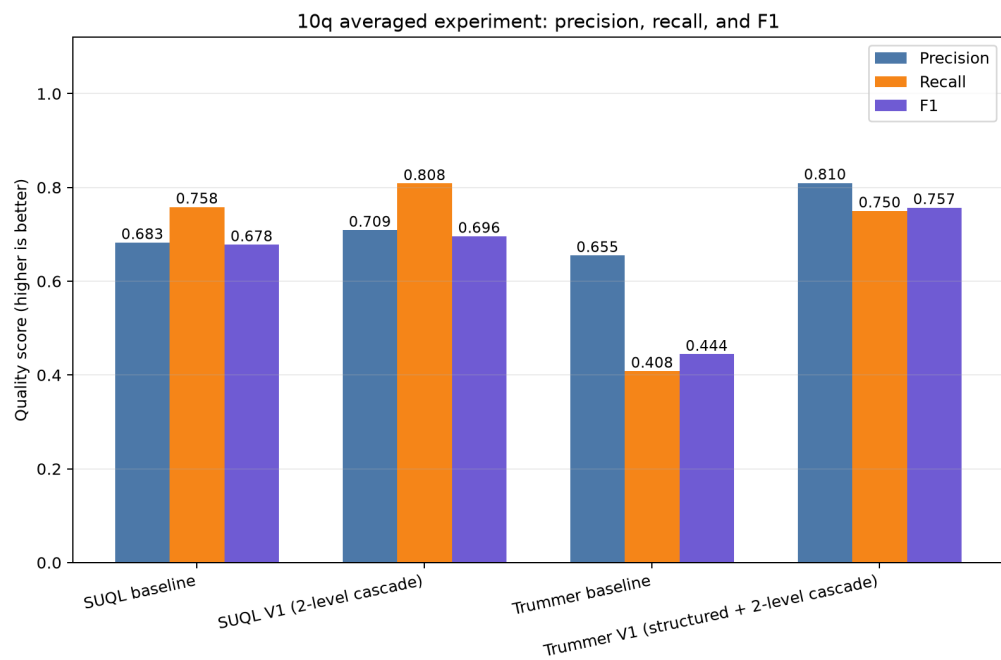


Figure 5.17: Qwen, Amazon five-question suite: precision, recall, F1 (error bars: std. dev. across repetitions).

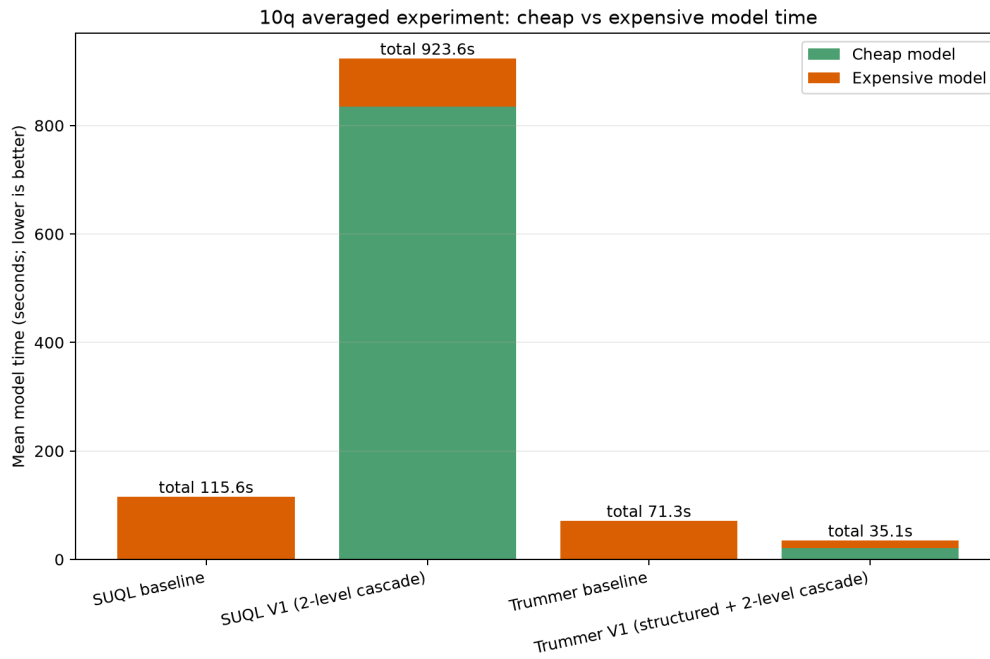


Figure 5.18: Qwen, Amazon five-question suite: mean wall time, cheap vs. expensive stage.

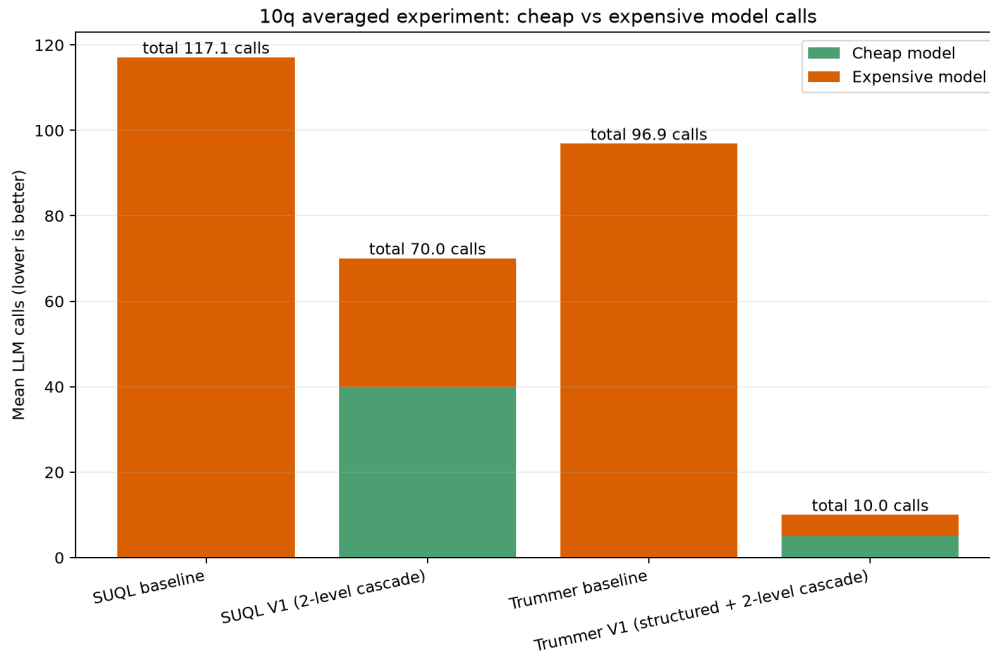


Figure 5.19: Qwen, Amazon five-question suite: mean LLM calls, cheap vs. expensive stage.

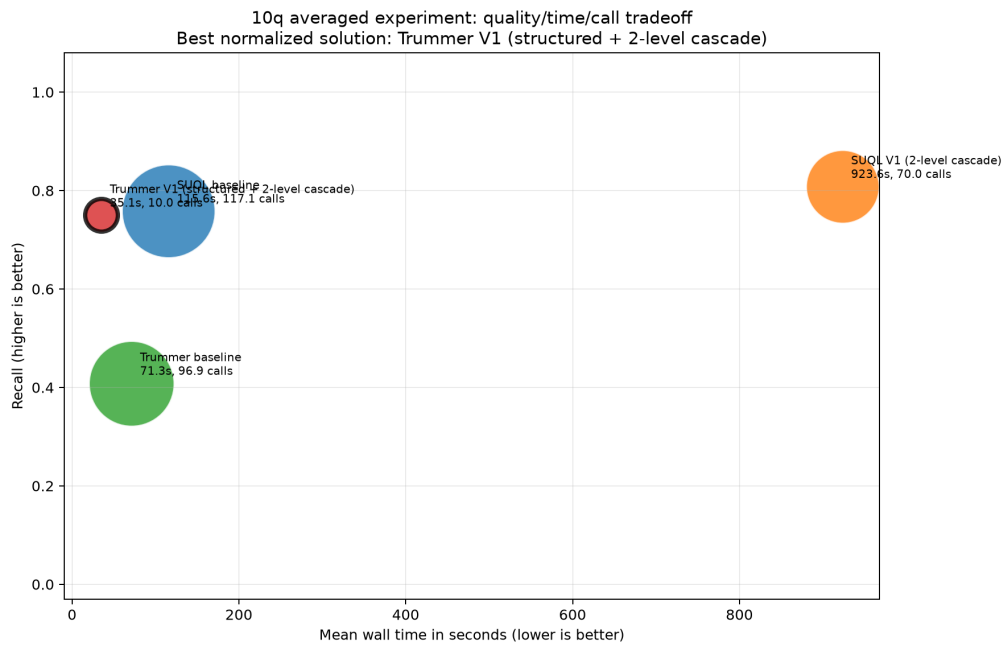


Figure 5.20: Qwen, Amazon five-question suite: recall vs. wall time, harness best-solution annotation is Trummer V1.

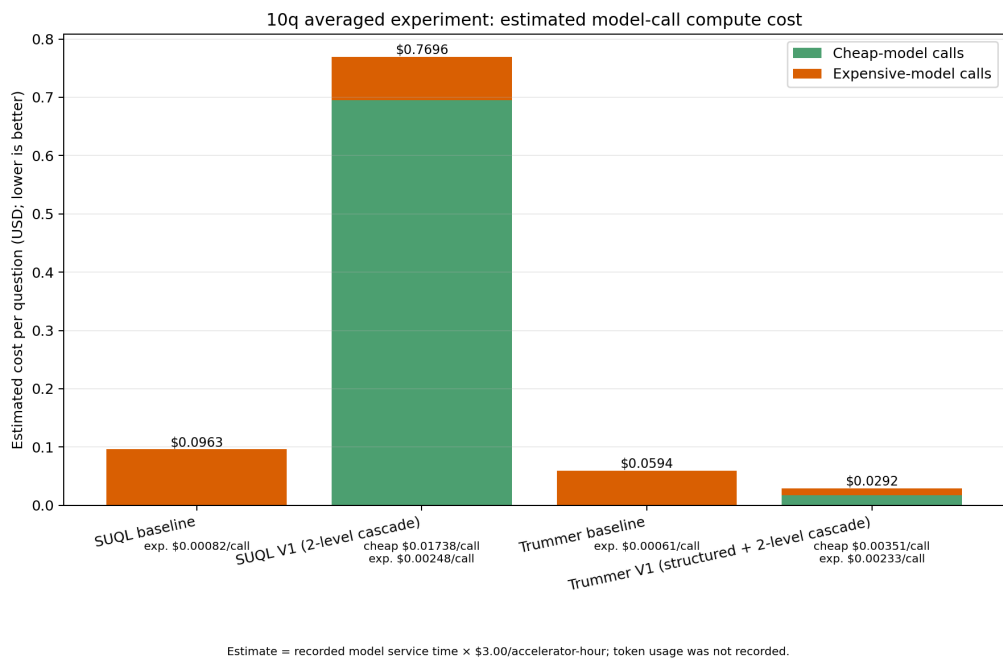


Figure 5.21: Qwen, Amazon five-question suite: estimated \$-cost per question.

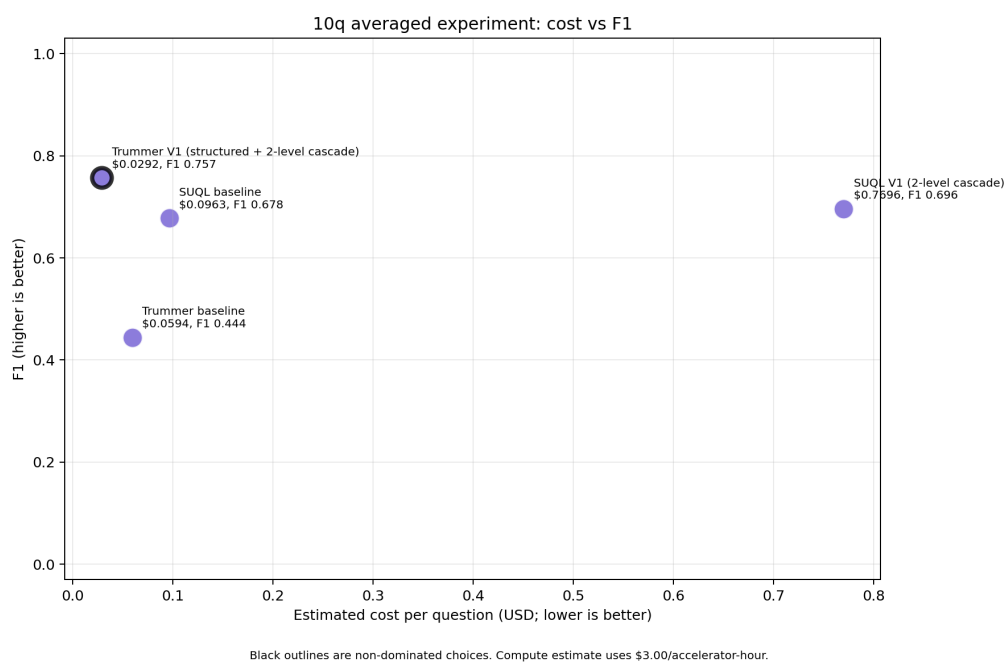


Figure 5.22: Qwen, Amazon five-question suite: cost–F1 Pareto frontier.

Conclusion and Future Work

6.1 Summary

This report set out to answer a concrete systems question: given a heterogeneous data pool mixing structured columns and free-text reviews, how should a query engine execute a natural-language question that mixes a structured predicate with a semantic one, while controlling both LLM invocation cost and wall-clock latency? We benchmarked two baseline strategies – row-wise predicate pushdown (SUQL [1]) and an adaptive, join-based batched strategy in the style of Trummer – and found them to occupy different points of the cost/quality trade-off space, now validated over ten held-out, genre-diverse questions rather than one (Chapter 5). We introduced a two-level cascade, applicable on top of either baseline, in which a cheap first-stage scorer resolves the majority of units of work immediately and escalates only genuinely ambiguous units to an expensive verification step. The cascaded batched design, Trummer V1, dominated on every axis we measured: best F1 (0.757), best precision (0.810), the highest *floor* across the ten questions (0.500, versus 0.267 or lower for every other method), and by far the lowest cost (10.0 calls, 35.05s per question) – at a small ($\approx 7\%$ relative) recall cost compared to the cascaded row-wise design. New in this revision, we also proved – rather than only observed – *why*: a calibrated cascade’s expected and (with high, exponentially-growing probability) realized cost is lower than an all-expensive baseline’s once the candidate pool is large enough, provided the cheap stage’s batching factor and calibrated escalation rate satisfy an explicit margin condition (Section 4.5). Plugging in the measured system constants shows SUQL V1 fails that margin condition on both calls and time (explaining its wall-time regression analytically, not just empirically), while Trummer V1 satisfies it comfortably on both.

6.2 Engineering Lessons

Beyond the headline numbers, four lessons from this project generalize beyond this specific workload:

1. **Predicate pushdown is the dominant cost lever** – confirmed, not just asserted, by the fact that the one method in the Trummer family that adds it (Trummer V1) is also the only one that is both fast *and* accurate.
2. **Batching the cheap stage is not an optimization detail – it determines whether cascading can win on cost or time at all.** Section 4.5’s worked instantiation shows this

precisely: SUQL V1’s unbatched ($B = 1$) cheap stage cannot satisfy the theorem’s margin condition under its current per-call latency, while Trummer V1’s batched ($B = 8$) cheap stage satisfies it with room to spare.

3. **Calibration conventions are a real source of bugs.** The α -as-confidence-level vs. α -as-tail-probability pitfall of Section 4.3.3 produced two individually defensible formulas, only one of which matched the calling code’s own convention; only a regression test against a synthetic confusion matrix with a known closed-form answer caught it.
4. **A single hand-verified query is not enough to trust a cost/quality comparison.** Moving from one query to the 10q suite changed the story in a material way: it surfaced Trummer V1’s floor-robustness (invisible in a single-query comparison, since there is no floor to speak of with $n = 1$) and it exposed a genuinely hard question (Q3, romance chemistry) on which two of the four methods nearly collapse – exactly the kind of finding a broader benchmark suite is for.

6.3 Current Implementation Status

At the time of writing, all four methods – SUQL baseline/V1, Trummer baseline/V1 – are implemented, documented, and benchmarked end to end in the public repository (<https://github.com/AnnaRemi/ENSIMAG-AI-M2-final-project>, branch `agent/consolidate-four-method-benchmarks`), with a shared runner and evaluator (`benchmarks/shared/scripts/`) producing the `aggregate.csv`, `comparison.csv`, and per-question plots this report draws on. Four benchmark suites of increasing size (1q, 3q, 5q, 10q) are available and runnable locally (against a local Ollama instance) or on the lab’s shared Aker cluster.

6.4 Future Work

6.4.1 Batch SUQL V1’s cheap stage

Section 4.5’s analysis makes this the single most consequential remaining change: SUQL V1’s cheap stage currently scores one row per call ($B = 1$), and its measured per-row latency already exceeds the expensive stage’s own per-call latency, which Proposition 4.2 shows makes a wall-time win *structurally impossible* regardless of calibration quality or pool size. Batching the cheap probe (scoring several rows’ log-odds in one completion, the same lever that already makes Trummer V1’s cheap stage effective) is predicted, by the same analysis, to be the change most likely to convert SUQL V1 from a quality-only cascade into one that also wins on cost and time.

6.4.2 Port Beta-posterior calibration to Trummer V1

As flagged in Section 4.2.2, Trummer V1’s threshold is currently learned by a point-estimate agreement search rather than the Beta-posterior credible-bound procedure used by SUQL V1 (Section 4.3). The two are not equally rigorous: only the Beta-posterior version currently carries an explicit, finite-sample confidence-level guarantee. Unifying both cascades on the

same calibration procedure would let Trummer V1 – already the strongest variant on every other axis – also carry that guarantee.

6.4.3 Amortize the calibration budget across repeated queries

Both cascades currently re-run their calibration procedure fresh for every question ($K_{\text{calls}} = 20$ row-wise calls for SUQL V1, 1 batched call for Trummer V1, per Table 4.1). In a deployment that re-asks a stable set of semantic predicates repeatedly (e.g. a fixed catalogue of recommendation filters), this cost is payable once and amortized over every future invocation of the same predicate, pushing the crossover point n^* of Proposition 4.2 even lower than the already-small value computed for Trummer V1 in Section 4.5.4.

6.4.4 The KV-cache-aware execution ladder

Stretto’s second contribution – *Expected Attention Press*, a cache-aware execution ladder that reuses attention key/value state across cascade stages – remains deliberately deferred. It requires direct access to the serving engine’s attention state (e.g. via `vllm`), which an Ollama-served backend does not expose; adopting it is future work contingent on a backend migration, and remains the single largest piece of remaining architectural work identified by this project.

6.4.5 Extending the selectivity/difficulty grid

The 10q suite fixes structural selectivity at 40% (Chapter 5); the repository’s smaller 1q, 3q, and 5q suites, and a still-larger suite beyond 10q, are the natural next steps toward a full grid crossing structural selectivity with semantic-predicate difficulty, which would let the cost/quality trade-offs of this report be stated as trends across that grid rather than at one fixed selectivity.

6.4.6 Early termination for LIMIT queries

As discussed in Chapter 5, wiring early stopping into the cascade’s main accept loop once $|Accept| \geq k$ for a LIMIT k query remains a small, self-contained, and still unimplemented optimization.

— A —

Prompts and Reproduction Reference

A.1 Complete Prompt Reference

For convenience, this section collects every prompt referenced in the body of the report in one place. All are transcribed verbatim (module docstrings/comments removed) from <https://github.com/AnnaRemi/ENSIMAG-AI-M2-final-project/tree/agent/consolidate-four-method-benchmarks>.

Listing A.1: SUQL baseline answer() system prompt (project SUQL/baseline/suql_engine.py).

```
You are a recall-first semantic filter for movie reviews.

Decide whether the review provides evidence that the movie
satisfies the question.

Recall is the primary objective, but do not accept every
review:
- Answer YES when there is any concrete review-specific
  evidence: direct wording, a synonym/paraphrase, a described
  example, or a reasonable implication.
- Give borderline evidence the benefit of the doubt when it
  specifically bears on the condition. The evidence need not
  use the question's exact words.
- This is evidence retrieval, not sentiment scoring: an
  explicit mention still counts when it is negated, qualified,
  quoted, or used critically.
- Answer NO when there is no review-specific support, the
  connection is only the movie's genre/topic, the text is
  generic praise or criticism, or the review never discusses
  the condition itself.
- Do not infer evidence merely because the condition is not
  contradicted.

Return exactly YES or NO. Do not include reasoning,
punctuation, or extra text.
```

Listing A.2: SUQL semantic-parser system prompt (project SUQL/baseline/suql_engine.py, PARSE_SYSTEM), which compiles the NL question

into a SUQL query.

```
You are a SUQL (Structured and Unstructured Query Language)
semantic parser.
SUQL extends SQL with two free-text functions:
  answer(free_text_column, 'question') -> 'Yes' | 'No' | short string
  summary(free_text_column) -> a short prose summary

{TABLE_SCHEMA}

Rules:
1. Use plain SQL WHERE clauses for ALL structured predicates
   (year, runtime, director, genres, title, movie_id).
2. Use answer() ONLY for predicates that require understanding
   the review text.
3. Always apply structured filters BEFORE answer() in the
   WHERE clause (efficiency).
4. The SELECT list must include: movie_id, title, year,
   runtime, director, genres. Add summary(review) AS
   review_summary whenever you use answer() on the review.
5. Use LIKE '%value%' for partial text matches on genres and
   director.
6. For "top N" questions without a numeric ranking field, use
   LIMIT N.
7. Output ONLY the raw SQL query -- no markdown fences, no
   explanation.

{FEW_SHOT_EXAMPLES}
```

Listing A.3: Trummer baseline block-join prompt (project Trummer/baseline/trummer_join/operators.py, create_prompt).

```
Find every movie in Collection 1 that has a review in
Collection 2 satisfying this predicate:
{predicate}
Act as a recall-first semantic filter, while requiring
review-specific evidence.
- Answer YES for direct wording, synonyms/paraphrases,
  described examples, or reasonable implications.
- Give borderline evidence the benefit of the doubt when it
  specifically bears on the predicate.
- Count explicit mentions even when negated, qualified,
  quoted, or critical; retrieve evidence rather than score
  sentiment.
- Answer NO when support is absent, generic, based only on
  genre/topic, or never discusses the predicate itself.
- Lack of contradiction alone is not evidence.
Collection 1 contains structured movie rows. Collection 2
contains review chunks.
A review belongs to a movie only when tconst is exactly
equal to movie_id.
Return one decision per line as: movie_id: YES or
movie_id: NO.
Do not include confidence, evidence, explanations, or prose.
Copy each movie_id exactly from Collection 1.
Do not add prose, JSON, markdown, or a completion marker.
```



```
Collection 1:
{block_1_rows}
```

```
Collection 2:
{block_2_rows}
```

```
Decisions:
```

Listing A.4: SUQL V1 cheap Stage 1 single-token scoring prompt (project SUQL/v1/scorer.py).

```
Act as a high-recall first-pass semantic filter. Decide
whether this review contains review-specific evidence for a
Yes answer. Count direct wording, synonyms, described
examples, and reasonable implications as evidence. Give
borderline but condition-specific evidence the benefit of
the doubt. Do not discard an explicit mention because it is
negated, qualified, quoted, or critical: this is evidence
retrieval, not sentiment scoring. Do not count genre/topic
alone, generic praise or criticism, or mere lack of
contradiction as evidence.
Return exactly one token: 1 for Yes, 0 for No.
```

```
Question: {question}
Review: {review[:1800]}
```

```
Answer:
```

Listing A.5: SUQL V1 expensive Stage 2 verification prompt (project SUQL/v1/cascade_filter.py, ANSWER_SYSTEM).

```
You are the final recall-first semantic filter for movie
reviews.
```

```
Decide whether the review provides evidence that the movie
satisfies the question.
```

```
Recall is the primary objective, but do not accept every
review:
```

- Answer YES for any concrete review-specific support, including direct wording, synonyms, described examples, and reasonable implications.
- Give borderline evidence the benefit of the doubt when it specifically bears on the condition; exact wording is unnecessary.
- This is evidence retrieval, not sentiment scoring: an explicit mention still counts when it is negated, qualified, quoted, or used critically.
- Answer NO when support is absent, merely generic, based only on genre/topic, or never discusses the condition itself. Lack of contradiction alone is not evidence.

```
Return exactly YES or NO. Do not include reasoning,
punctuation, or extra text.
```

Listing A.6: Trummer V1 cheap Stage 1 batched prompt (project Trummer/v1/trummer_join/cascade.py, _cheap_batch_prompt).

```
Classify every explicit candidate pair below.
Predicate: {predicate}
{CHEAP_EVIDENCE_INSTRUCTIONS}
Return one line per candidate as CANDIDATE_ID: YES, NO, or
UNCERTAIN.
Do not include confidence, evidence, explanations, or prose.
Return every candidate exactly once and do not explain.

CANDIDATE_{n}:
Movie: {movie_text}
Review: {review_text}
...

Decisions:
```

Listing A.7: Trummer V1 expensive Stage 2 batched verification prompt (cascade.py, _expensive_batch_prompt).

```
Evaluate the explicit candidate pairs below.
Predicate: {predicate}
{EXPENSIVE_EVIDENCE_INSTRUCTIONS}
Return one line per candidate as PAIR_ID: YES or PAIR_ID: NO.
Do not include confidence, evidence, explanations, or prose.
Return every candidate exactly once. Do not add prose, JSON,
or markdown.

PAIR_{n}:
Movie: {movie_text}
Review: {review_text}
...

Decisions:
```

A.2 Calibration Reference Values

Table A.1 lists the calibrated cheap-stage accept/reject log-odds thresholds actually stored in project SUQL/v1/thresholds.json for a representative sample of predicates from earlier (pre-10q) calibration runs; the 10q benchmark of Chapter 5 recalibrates fresh per question rather than reusing this file, but the values illustrate the typical magnitude of the calibrated band.

A.3 Reproduction: Running the Benchmarks Locally

The repository’s runner supports all four methods and all four suite sizes out of the box; the commands below are the actual entry points (not a hypothetical interface), taken from the repository’s top-level and benchmarks/ README files.

Predicate (abbreviated)	Accept τ_+	Reject τ_-
Funny, humorous, witty, or entertaining?	1.0	−1.0
Praise acting/performances/cast/characters?	1.0	−1.0
Suspense, tension, scares, fear, creepy atmosphere?	1.0	−1.0
Praise chemistry, romance, love story, relationships?	1.0	−1.0
Exciting, thrilling, intense, action-packed?	1.0	−1.0

Table A.1: Sample calibrated thresholds, project SUQL/v1/thresholds.json. The symmetric ± 1.0 band corresponds to routing everything with cheap-stage probability outside roughly $[0.27, 0.73]$ directly, and escalating the rest.

Listing A.8: Cloning and running the smoke-test suite locally against Ollama.

```
git clone git@github.com:AnnaRemi/ENSIMAG-AI-M2-final-project.git
cd ENSIMAG-AI-M2-final-project
bash benchmarks/1q/run_local.sh --pull-models
```

Listing A.9: Running the full 10-question, 10-repetition suite on the lab’s Aker cluster (the exact configuration used to produce Chapter 5’s tables).

```
bash benchmarks/run_aker.sh \
  --suite 10q \
  --repetitions 10 \
  --methods "suql_baseline suql_v1 trummer_baseline trummer_v1" \
  --cheap-model gemma4:e2b \
  --expensive-model gemma4:26b \
  --pull-models
```

Both entry points create a .venv, install the shared Python dependencies, pull the configured models, verify the local Ollama API, and run all four implementations end to end, writing aggregate.csv, comparison.csv, and the four aggregate plots (quality, time, call-count, and a speedup-vs-call-reduction scatter, plus per-question versions of the same four plots) to benchmarks/<suite>/outputs/<run-name>/. Raw model responses and per-repetition intermediate artifacts are deleted after successful evaluation by default (pass -keep-run-artifacts to shared/scripts/run_all.py directly to retain them for debugging).

Bibliography

- [1] Shicheng Liu et al. Suql: Conversational search over structured and unstructured data with large language models, 2023.
- [2] G. Sanmartino et al. Stretto: Bayesian threshold calibration and cache-aware cascades for llm query operators, 2025.
- [3] Matthias Urban and Carsten Binnig. Eleet: Efficient learned query execution over text and tables. *Proceedings of the VLDB Endowment (PVLDB)*, 2024.